CPE477/ECG677 – Embedded Security & Machine Learning

Design Assignment 2

Decision Tree Model Comparison for Activity Recognition

DO NOT REMOVE THIS PAGE DURING SUBMISSION:

Name: [your name]

Group-Partner Name: [partner name, or N/A]

Email: [your email]

Github Repository link (root): [paste the link after creating the private repository]

Youtube Playlist link (root): [paste the link of the EMBSEC&ML playlist]

Follow the submission guideline to be awarded points for this Assignment.

Submit the following for all Assignments:

1. In the document, for each task submit the modified or included code (from the base code) with highlights and justifications of the modifications. Also include the comments. If no base code is provided, submit the base code for the first task only.
2. Create a private Github repository with a random name (no CPE477/677, Lastname, Firstname). Place all assignments under the root folder, sub-folder named Assignmentn, with one document and one video link file for each lab, place modified c files named as main.c.
3. If multiple 'c' or 'h' files or other libraries are used, place these files inside the folder.
4. The folder should have a) Word document (see template), b) source code file(s) with other 'c' and 'h' include files, c) text file with youtube video links (see template).
5. Submit the pdf file in canvas before the due date. The root folder of the github assignment directory should have the documentation and the text file with youtube video links.
6. Organize your youtube videos as playlist under the name "EMBSEC&ML". The playlist should have the video sequence arranged as submission or due dates.
7. Only submit pdf documents. Do not forget to upload this document in the github repository and in the canvas submission portal.

Declaration

I understand the Student Academic Misconduct Policy - http://studentconduct.unlv.edu/misconduct/policy.html

"This assignment submission is my own, original work".

Name of the Student: [type your name yourself to sign this declaration]

1. Abstract

The goal of this design assignment was to develop, train and compare four decision-tree classifiers for human-activity recognition with the LSM6DSV16X IMU on a NUCLEO-F401RE board with the X-NUCLEO-IKS4A1 shield: a tree on the six raw IMU values (RAW), a tree on windowed statistics of the six axes (RAW+FE), a tree on the four quaternion components (QUAT), and a tree on windowed quaternion statistics chosen by ANOVA and recursive feature elimination (QUAT+FE). About 29 s each of Static, Horizontal and Vertical motion were recorded with MEMS Studio, and the models were trained in Python (scikit-learn) on a chronological 70/15/15 split with identical hyperparameters, exported as JSON and as C code (checked against the Python models), and compared for accuracy, size and cost. The IMU models were accurate (RAW 99.4 %, RAW+FE 100 % on 41 test windows with a tree of depth 2), whereas the quaternion models reached only 75.5 % (QUAT) and 80.5 % (QUAT+FE), because each activity was recorded at a different and drifting orientation; with motion-only quaternion statistics the window model again reached 100 % on the same test windows.

2. Schematics

The hardware is a NUCLEO-F401RE board with the X-NUCLEO-IKS4A1 expansion board plugged on its Arduino UNO R3 connectors. The LSM6DSV16X on the shield is read over the I2C1 bus of the STM32F401RE, and the data reach the PC over the USB cable of the on-board ST-LINK/V2-1 as a virtual COM port (COM4 in MEMS Studio). Figure S1 shows the setup and Table S1 lists every signal, pin and its function.

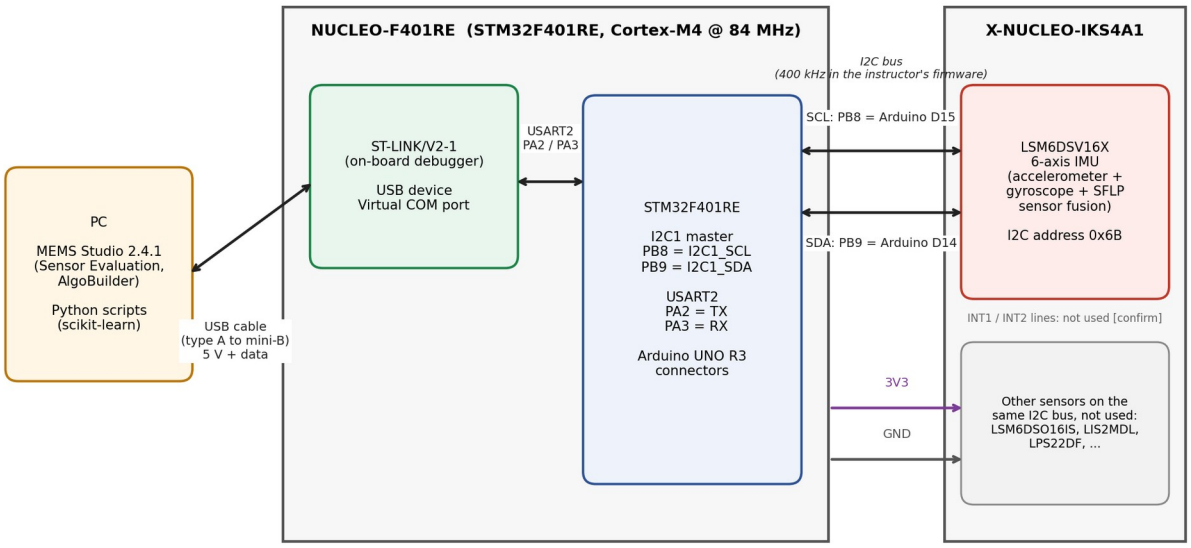


Figure S1. Hardware setup: wiring and signals.

Table S1. Connections, pins and functions

Signal	From → to	Pin / port	Function
USB cable	PC ↔ ST-LINK/V2-1	USB type A to mini-B	Powers the board (5 V) and carries the data; the virtual COM port appears as COM4 in MEMS Studio.
USART2 TX / RX	STM32F401RE ↔ ST-LINK/V2-1	PA2 (TX), PA3 (RX)	Virtual COM port to the PC. The instructor's firmware uses 921600 baud, 8-N-1.
I2C1 SCL	STM32F401RE → LSM6DSV16X	PB8 = Arduino D15	Clock of the I2C bus (400 kHz fast mode in the instructor's firmware).
I2C1 SDA	STM32F401RE ↔ LSM6DSV16X	PB9 = Arduino D14	Bidirectional data line of the I2C bus; the IMU is read by polling.
I2C address	on the I2C bus	0x6B (7-bit)	Selects the LSM6DSV16X in the standard I2C mode of the shield (Zephyr shield overlay; see UM3239).
3V3	NUCLEO Arduino power header → shield	3V3	Supply of the shield and the sensors.
GND	NUCLEO ↔ shield	GND	Common ground.
INT1 / INT2	LSM6DSV16X → Arduino header	INT2 on D4 (PB5) in the Zephyr overlay	Interrupt / data-ready outputs of the IMU; not used because the data are read by polling [confirm].

Table S2. Experimental setup (software and settings)

Item	Value
Host software	MEMS Studio 2.4.1 (Sensor Evaluation, Advanced Features / MLC, AlgoBuilder); Python 3 with pandas, NumPy, Matplotlib, scikit-learn
Board firmware	Sensor Evaluation firmware 20.4.1; AlgoBuilder firmware 10.0.0 (as shown in the status bar of MEMS Studio)
Accelerometer / gyroscope range	±8 g [confirm] / ±2000 dps
Output data rate	[confirm: nominal ODR you configured]; effective logging rate about 71 Hz
Recorded data	Static, Horizontal, Vertical; 2,038 / 2,100 / 2,135 samples (about 29 s each), logged with time, acceleration (mg), angular rate (mdps) and quaternion q0–q3

Pin names follow the NUCLEO-F401RE user manual UM1724 (Arduino D14 = PB9, D15 = PB8; USART2 on PA2/PA3 through the ST-LINK virtual COM port) and the X-NUCLEO-IKS4A1 user manual UM3239. Items marked [confirm] must be checked against your own firmware settings.

3. Task Results

Each task below gives its experimental setup, the data, the visualisations, the analysis and the outcome, including issues and shortcomings. Table and figure numbers run through the whole document. The comparison of each task with the previous one is in Section 4.

Task 1: Data Preparation and Exploratory Analysis

Overview

This section documents how the IMU dataset for the activity-recognition task was collected, cleaned and explored, and how it was split for training, validation and testing. Three activity classes were recorded from the LSM6DSV16X 6-axis IMU on an X-NUCLEO-IKS4A1 expansion board connected to a NUCLEO-F401RE: Static, HMotion (horizontal motion) and VMotion (vertical motion). Each recording contains the accelerometer (Acc_X, Acc_Y, Acc_Z), the gyroscope (Gyro_X, Gyro_Y, Gyro_Z) and the quaternion (q0, q1, q2, q3) from the on-board 6-axis sensor-fusion block. All numbers, tables and figures below are produced by the accompanying script data_preparation.py.

Sensor configuration

The LSM6DSV16X defaults (±2 g accelerometer, accelerometer powered down) are not suitable for this task. The settings below were used for the recordings.

Table 1. Sensor settings

Parameter	Setting	Rationale
Accelerometer full scale	±8 g [confirm]	Hand motion produced peaks of about 3.0 g (VMotion acc_z), which would clip at ±2 g. ±8 g leaves headroom.
Gyroscope full scale	±2000 dps	Measured peaks reached about 170 dps. The logged gyroscope values move in steps of 70 mdps, which is the resolution of the ±2000 dps range.
Output data rate	[confirm: e.g. 100 Hz request]	Human motion is mostly below 20 Hz, so the sampling rate used (see below) is far above the Nyquist requirement.
Sensor fusion	6-axis (accelerometer + gyroscope)	AlgoBuilder “Quaternions 6X” block. There is no magnetometer, so yaw is relative to the starting heading.

Check the bracketed values against your Quick Setup / firmware settings before submitting.

The log timestamps lie on a 10 ms grid, but about 36–45 % of the steps are 20 ms, so the samples are not evenly spaced and the **effective sampling rate is about 69–73 Hz** (Table 7). The analysis therefore always uses the logged timestamps instead of assuming a fixed rate.

Data collection

Each activity was recorded as a separate file of about 29 s (2,038 to 2,135 rows) and labelled from the file name (Static.csv, Horizontal.csv, Vertical.csv).

Table 2. Activity definitions

Class	File	Definition
Static	Static.csv	Board resting still on a flat surface.
HMotion	Horizontal.csv	Board moved back and forth horizontally along its x axis at about 2 Hz [describe exactly: distance, how it was held].
VMotion	Vertical.csv	Board moved up and down vertically at about 2 Hz [describe exactly: amplitude, how it was held].

Fill in the bracketed details and state where and how the board was held, so the motions can be reproduced.

Pre-processing

The following steps are applied in data_preparation.py:

- **Column detection and units.** The logged columns are time[us], acc[mg], gyro[mdps] and q0–q3. Acceleration is converted to g, angular rate to dps, and time to seconds from the start of each file.
- **Class labels.** The label comes from the file name (horizontal → hmotion, vertical → vmotion, static → static).
- **Trimming.** The first and last 2 s of each recording are removed to discard the moments of pressing Start and Stop. This leaves about 25 s per class.
- **Validity checks.** A file is rejected if a sensor channel is entirely zero (a powered-down sensor logs zeros). All three files passed: there are no all-zero rows, no repeated timestamps, and the quaternion norm stays between 0.999 and 1.001.
- **Quaternions.** q0–q3 are taken from the file, where they were produced on the board by the 6-axis fusion algorithm. No sign change or reordering is applied. When the board lies flat, the output is about (0.006, 0.000, 0.997, −0.078), so q2 is close to 1. [Confirm the component order (which column is the scalar part) in the firmware that wrote the log, and state it here.]
- **No resampling of the dataset.** The samples are kept as logged, so no synthetic samples are created. Only the frequency analysis (temporal dynamics) interpolates the signal onto a uniform 100 Hz grid, because an FFT needs evenly spaced samples.

Exploratory data analysis

Time-series signals

Figure 1 shows a 5-second excerpt of each class for all six IMU axes and the quaternion. Figures 2–4 show the full recordings.

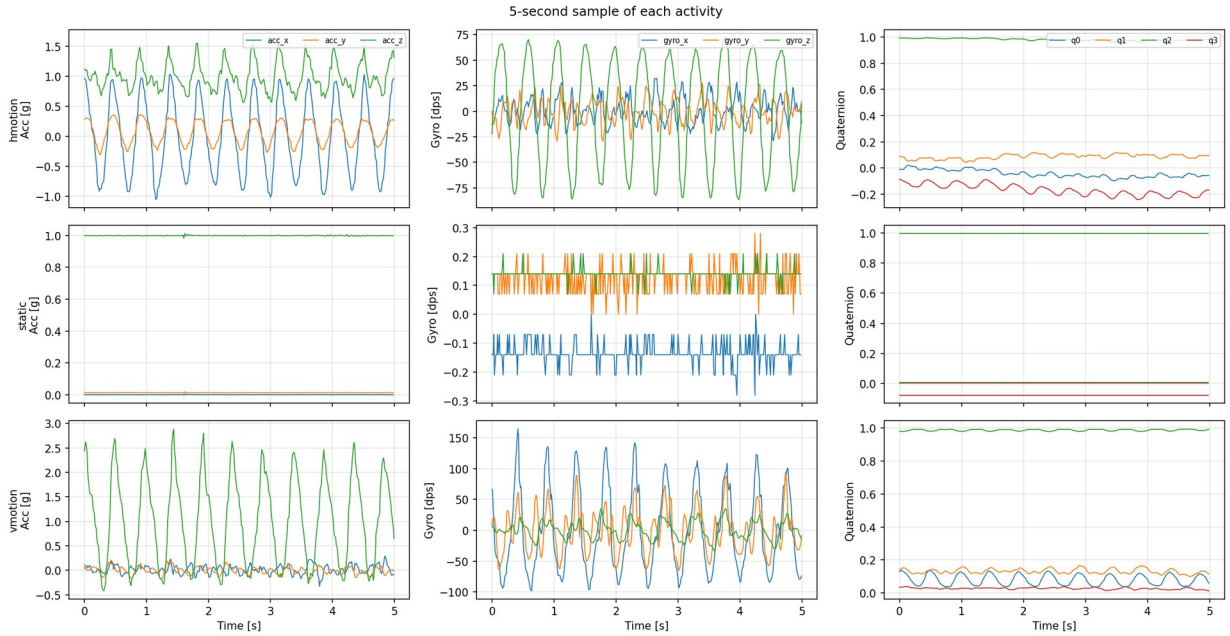


Figure 1. Five-second sample of each activity: accelerometer (left), gyroscope (centre) and quaternion (right).

Static. All axes are flat: acc_z ≈ 1 g (gravity) and acc_x, acc_y ≈ 0. The gyroscope moves only in 0.07 dps steps, which is the sensor's quantisation and noise floor.

HMotion. Regular oscillation at about 2.3 Hz, mainly in acc_x (about −1.2 g to +1.2 g) with a strong gyro_z rotation (about ±90 dps). acc_z swings moderately (0.5 to 1.6 g) and acc_y only slightly.

VMotion. Regular oscillation at about 2 Hz, mainly in acc_z (from about −0.6 g to 3.0 g), with gyro_x and gyro_y oscillating up to about ±100 to 170 dps. acc_x and acc_y stay small.

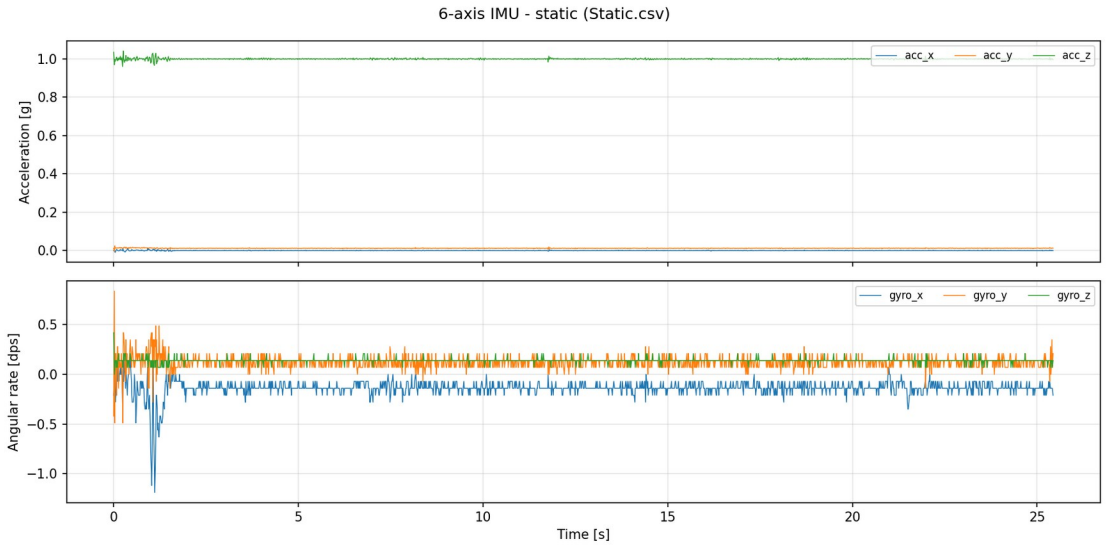


Figure 2. Static: full recording.

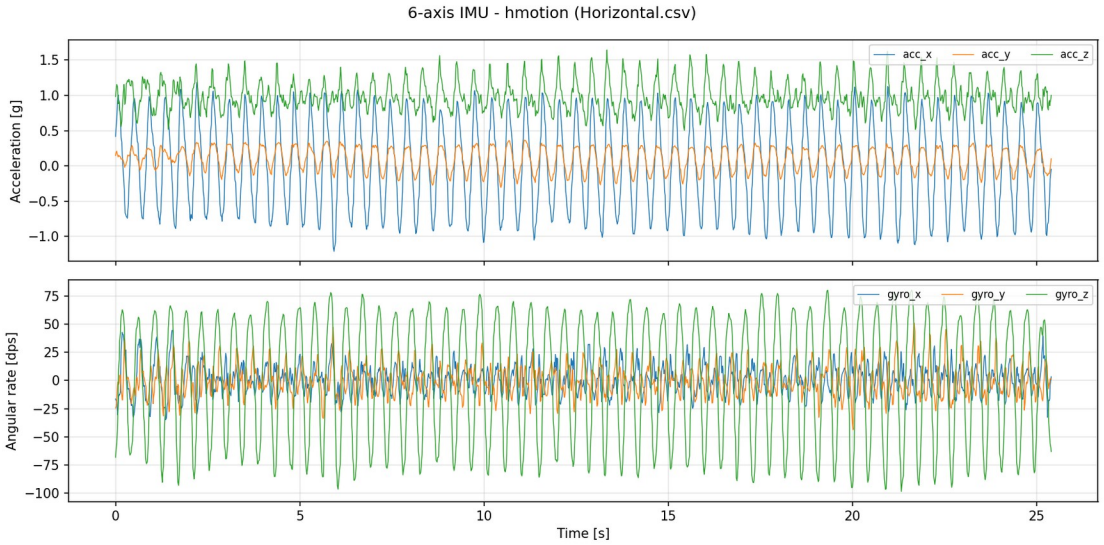


Figure 3. HMotion: full recording.

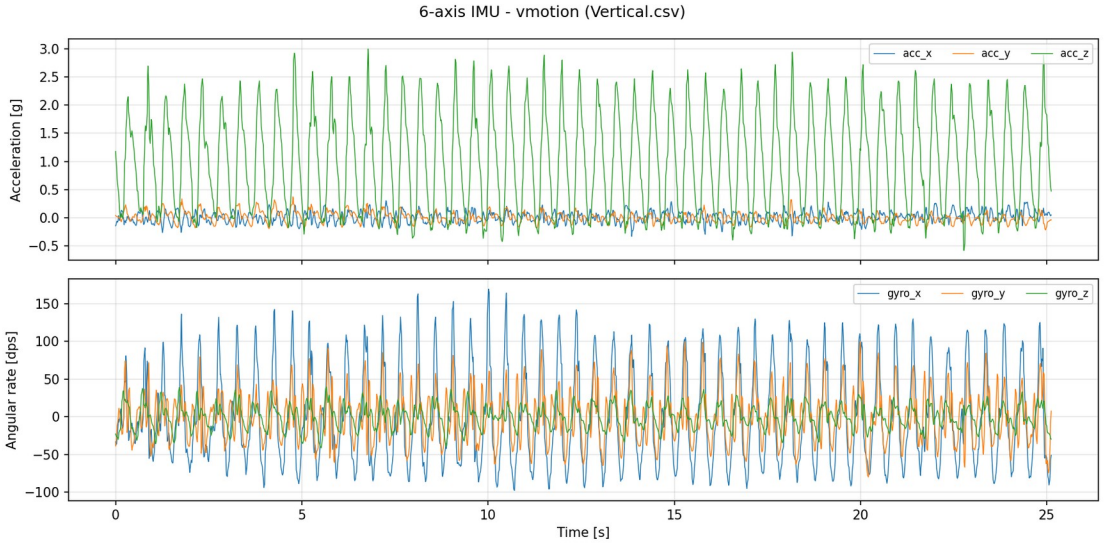


Figure 4. VMotion: full recording.

Per-class statistics

Table 3 lists the mean, standard deviation, minimum and maximum of every feature per class (acceleration in g, angular rate in dps, quaternion components unitless).

Table 3. Per-class statistics

Feature	Class	Mean	Std	Min	Max
acc_x	static	0.000	0.001	-0.009	0.012
acc_x	hmotion	0.014	0.633	-1.211	1.186
acc_x	vmotion	0.016	0.095	-0.328	0.304
acc_y	static	0.013	0.001	-0.004	0.025
acc_y	hmotion	0.091	0.168	-0.304	0.364

Feature	Class	Mean	Std	Min	Max
acc_y	vmotion	0.006	0.093	-0.214	0.373
acc_z	static	0.999	0.004	0.960	1.041
acc_z	hmotion	0.998	0.205	0.503	1.643
acc_z	vmotion	1.000	0.870	-0.575	2.994
gyro_x	static	-0.138	0.084	-1.190	0.210
gyro_x	hmotion	-0.113	13.062	-34.860	44.450
gyro_x	vmotion	-0.538	62.006	-97.860	169.260
gyro_y	static	0.123	0.068	-0.490	0.840
gyro_y	hmotion	0.191	13.542	-43.680	51.030
gyro_y	vmotion	-0.115	33.817	-80.010	99.470
gyro_z	static	0.140	0.021	0.000	0.420
gyro_z	hmotion	-0.222	53.338	-98.630	80.360
gyro_z	vmotion	0.045	15.767	-41.440	39.550
q0	static	0.006	0.000	0.006	0.007
q0	hmotion	0.002	0.039	-0.097	0.077
q0	vmotion	0.084	0.032	0.028	0.170
q1	static	-0.000	0.000	-0.001	0.001
q1	hmotion	0.064	0.027	-0.025	0.118
q1	vmotion	0.134	0.017	0.083	0.174
q2	static	0.997	0.000	0.997	0.997
q2	hmotion	0.987	0.009	0.960	0.998
q2	vmotion	0.986	0.005	0.973	0.995
q3	static	-0.078	0.000	-0.078	-0.078
q3	hmotion	-0.131	0.048	-0.245	-0.032
q3	vmotion	0.029	0.020	-0.032	0.079

Observations:

- Static is separable from both motion classes by variance alone: every accelerometer axis has a standard deviation of at most 0.004 g and every gyroscope axis at most 0.09 dps, hundreds of times smaller than for the moving classes.
- HMotion and VMotion put their energy on different axes. HMotion has a large acc_x spread (std 0.63 g, versus 0.10 g for VMotion) and gyro_z (std 53 dps, versus 16 dps). VMotion has a large acc_z spread (std 0.87 g, versus 0.21 g for HMotion) and large gyro_x and gyro_y (std 62 and 34 dps, versus 13 and 14 dps).
- The quaternion is dominated by q2 (about 0.99 for every class), because the board stayed close to flat. The remaining components, however, differ between classes by more than their spread within a class: the class means of q1 are 0.000 (Static), 0.064 (HMotion) and 0.134 (VMotion), and those of q3 are −0.078, −0.131 and +0.029. These constant offsets probably reflect the starting orientation and heading of each recording and how the board was held, and not the motion itself.

A model that uses only quaternions could therefore separate these classes for reasons unrelated to the activity (relevant for Task 4).

Feature ranges

Table 4. Feature ranges over the whole dataset

Feature	Unit	Min	Max	Mean	Std
acc_x	g	-1.211	1.186	0.010	0.371
acc_y	g	-0.304	0.373	0.037	0.118
acc_z	g	-0.575	2.994	0.999	0.521
gyro_x	dps	-97.860	169.260	-0.266	36.950
gyro_y	dps	-80.010	99.470	0.065	21.226
gyro_z	dps	-98.630	80.360	-0.014	32.215
q0	-	-0.097	0.170	0.031	0.048
q1	-	-0.025	0.174	0.067	0.058
q2	-	0.960	0.998	0.990	0.007
q3	-	-0.245	0.079	-0.059	0.074

All values lie well inside the configured full scales, so no channel saturated. The accelerometer spans −0.6 g to +3.0 g and the gyroscope −98 to +169 dps. The quaternion components stay within [−0.25, 1.0] and have unit norm.

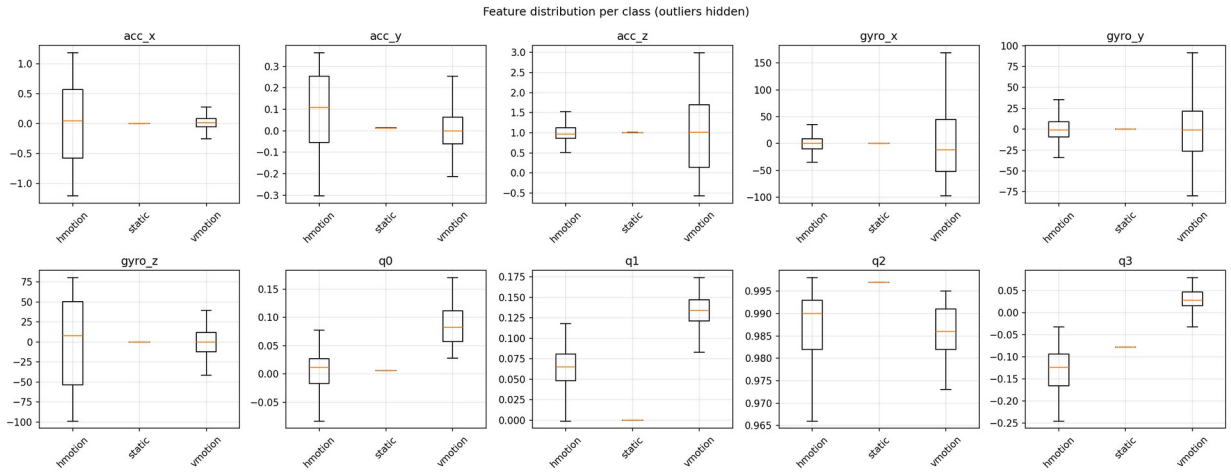


Figure 5. Distribution of every feature per class (outliers hidden).

Temporal dynamics

Table 5. Dynamics of the acceleration and angular-rate magnitude

Class	Acc std [g]	Acc peak freq [Hz]	Gyro std [dps]	Gyro peak freq [Hz]
hmotion	0.233	2.34	21.4	4.30
static	0.003	11.72	0.1	0.78
vmotion	0.790	1.95	34.2	4.30

The magnitude of each signal was interpolated onto a uniform 100 Hz grid and its spectrum estimated with Welch's method. HMotion peaks at 2.34 Hz and VMotion at 1.95 Hz in the acceleration magnitude (Figure 6), matching the roughly 2 Hz cycles in Figures 3 and 4. The gyroscope magnitude of both classes peaks at 4.30 Hz, which is twice the motion frequency (the magnitude is unsigned, so each swing in either direction counts as one cycle). The frequencies listed for Static (11.7 Hz and 0.8 Hz) are noise: the signal level is only 0.003 g, so Static has no periodic content.

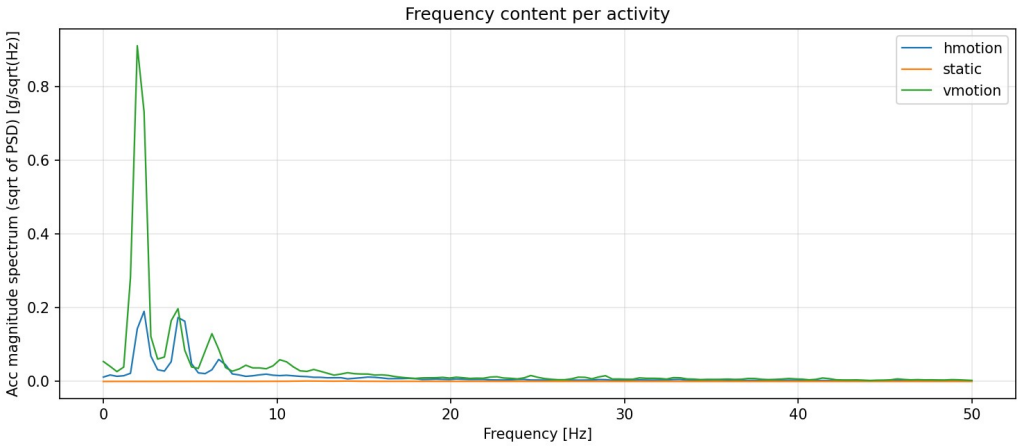


Figure 6. Spectrum of the acceleration magnitude per class.

Dataset characteristics and class distribution

Table 6. Dataset summary after trimming

Class	Samples	Duration [s]	Share [%]
hmotion	1814	25.4	33.5
static	1756	25.4	32.4
vmotion	1842	25.1	34.0

The dataset contains **5,412 samples** (about 76 s) in three classes, with 6 raw IMU features plus 4 quaternion components per sample. The classes are balanced at 32–34 % each (Figure 7), so no resampling or class weighting is needed.

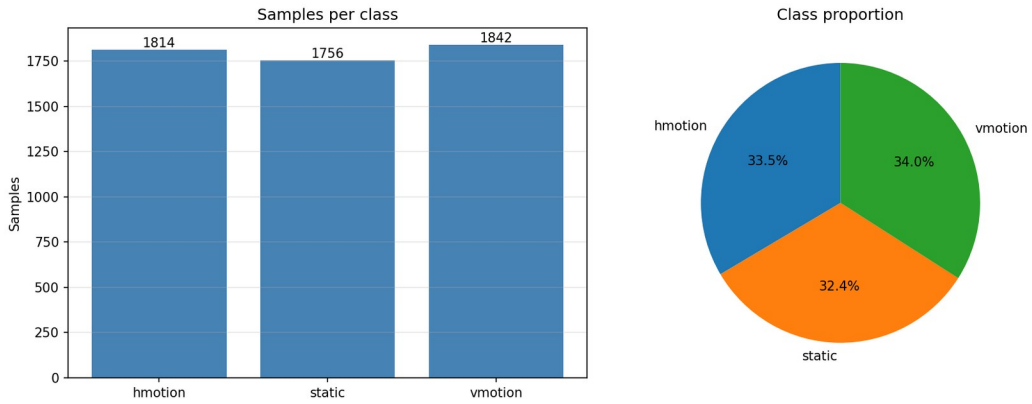


Figure 7. Class distribution (bar and pie chart).

Table 7. Sampling check per recording

Class	Samples	Duration [s]	Rate [Hz]	Step min / max [ms]	Steps of 20 ms [%]
hmotion	1814	25.4	71.4	10 / 20	40
static	1756	25.4	69.0	10 / 20	45
vmotion	1842	25.1	73.3	10 / 20	36

Stratified train / validation / test split

The samples were split with scikit-learn's train_test_split, stratified by class, in two stages (70 % train, then the remaining 30 % divided equally into validation and test) with a fixed random seed of 42. The same saved split files (train.csv, val.csv, test.csv) are used by every later task so that all models are compared on identical data.

Table 8. Samples per class in each set

Class	Train	Val	Test	Total	Train %	Val %	Test %
hmotion	1270	272	272	1814	70.0	15.0	15.0
static	1229	263	264	1756	70.0	15.0	15.0
vmotion	1289	277	276	1842	70.0	15.0	15.0
ALL	3788	812	812	5412	70.0	15.0	15.0

Every class is split in almost exactly 70/15/15, confirming that the stratification worked.

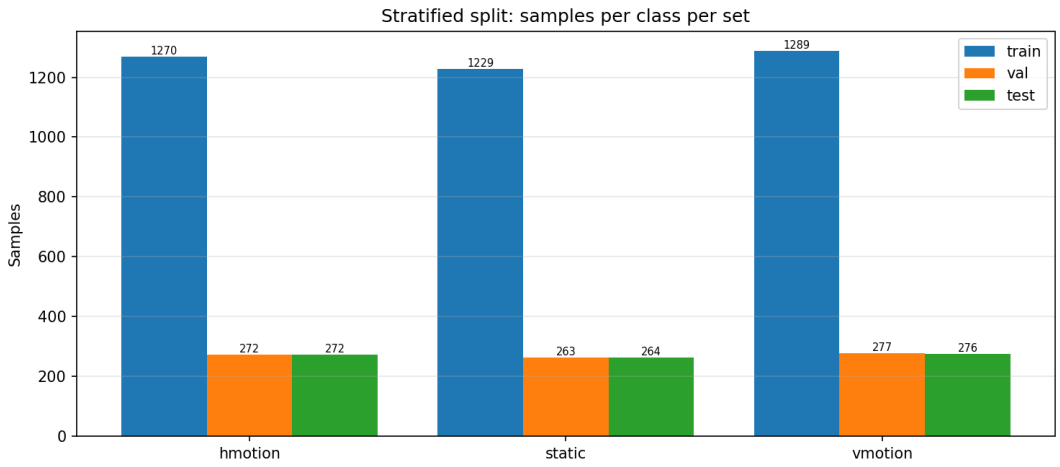


Figure 8. Samples per class in the train, validation and test sets.

Limitations

- **Small dataset.** Each class has about 25 s of data from a single recording. The validation and test sets have about 270 samples per class.
- **Optimistic evaluation.** At about 70 Hz and a 2 Hz motion, consecutive samples are almost identical. Because the split is random per sample, near-duplicates of a test sample are present in the training set, which inflates test accuracy. A fairer test would hold out entire recordings, which requires several recordings per class.
- **Irregular sampling.** The timestamps alternate between 10 ms and 20 ms steps. Windowed features (Task 3) should therefore be computed over time windows (for example 1.5 s), and not over a fixed number of samples.
- **Quaternion offsets.** The class-specific quaternion offsets (see the per-class statistics) probably come from the start orientation of each recording. Without a magnetometer, heading is not observable, so these offsets are not a reliable property of the activity.
- **Single subject and placement.** All data come from one person and one board orientation, so results will not necessarily generalise to other users or mounting positions.

Code: data_preparation.py

The script is run as python data_preparation.py (CSV files in data/raw/) and contains the following functions:

- **load_recording / load_dataset** – read the CSVs, detect columns, convert units, label, validate, trim edges and (if a file has no quaternion) compute it with a Madgwick filter.
- **sampling_report, per_class_statistics, feature_ranges, dominant_frequencies** – statistics and timing checks (Tables 3–7).
- **plot_full_timeseries, plot_activity_samples, plot_class_distribution, plot_feature_boxplots, plot_spectra, plot_split** – all figures.
- **stratified_split** – 70/15/15 stratified split; writes train.csv, val.csv and test.csv to data/processed/.

Key snippets (the complete script is submitted with this report):

1. Labels and trimming

```
LABEL_MAP = {"horizontal": "hmotion", "vertical": "vmotion", "static": "static"}
word = re.split(r"[_.\-\s]", path.stem)[0].lower()
df["label"] = LABEL_MAP.get(word, word)

# Remove the first/last 2 s of every recording (pressing Start / Stop)
keep = (df["t"] >= TRIM_SECONDS) & (df["t"] <= df["t"].iloc[-1] - TRIM_SECONDS)
df = df[keep].copy()
```

2. Uniform grid for the frequency analysis (timestamps are irregular)

```
def _uniform_signals(g, cols):
    out = []
    for _, f in g.groupby("source_file"):
        t = f["t"].to_numpy()
        grid = np.arange(0, t[-1], 1 / RESAMPLE_HZ)          # 100 Hz
        comp = [np.interp(grid, t, f[c].to_numpy()) for c in cols]
        out.append(np.linalg.norm(np.vstack(comp), axis=0))   # magnitude
    return out
```

3. Stratified 70 / 15 / 15 split

```
VAL_FRAC, TEST_FRAC, RANDOM_STATE = 0.15, 0.15, 42

def stratified_split(data):
    train, temp = train_test_split(                  # 70 % train, 30 % held out
        data, test_size=VAL_FRAC + TEST_FRAC,
        stratify=data["label"], random_state=RANDOM_STATE)
    val, test = train_test_split(                    # held-out 30 % -> 15 % + 15 %
        temp, test_size=TEST_FRAC / (VAL_FRAC + TEST_FRAC),
        stratify=temp["label"], random_state=RANDOM_STATE)
    return train, val, test
```

Task 2: RAW Model – 6-Axis Raw Features

Approach

A decision tree classifier was trained on the six raw IMU values of each sample only: Acc_X, Acc_Y, Acc_Z and Gyro_X, Gyro_Y, Gyro_Z. No windowing, scaling or engineered feature is used, so every sample is classified on its own as **Horizontal** (HMotion in Task 1), **Static** or **Vertical** (VMotion). The values keep the units of the MEMS Studio log (mg and mdps), which is also what the generated C code expects. The model is trained with the script model_raw.py (scikit-learn DecisionTreeClassifier), which also exports the tree as JSON and as C code for the board.

Data split

Each recording is split chronologically: the first 70 % of its samples are used for training, the next 15 % for validation and the last 15 % for testing. Every class therefore keeps the same 70/15/15 proportions, and neighbouring samples (which are almost identical at about 71 samples per second) are not spread over several sets, as they would be in a random split. This is stricter than the stratified random split used for the exploratory dataset in Task 1. The first and last seconds of each recording are not trimmed in this task.

Table 9. Samples per class in each set

Class	Train	Validation	Test	Total
Horizontal	1470	315	315	2100
Static	1426	306	306	2038
Vertical	1494	320	321	2135
All	4390	941	942	6273

Hyperparameters

Table 10. Model configuration (to be kept identical in Tasks 3, 4 and 5)

Hyperparameter	Value
criterion	gini
max_depth	None (the tree grows until the leaves are pure)
min_samples_leaf	1
min_samples_split	2
random_state	42

```
FEATURES = ["ACC_X", "ACC_Y", "ACC_Z", "GYR_X", "GYR_Y", "GYR_Z"]

# chronological split of every recording: 70 % train, 15 % validation, 15 % test
train_end = int(n * 0.70);  validation_end = int(n * (0.70 + 0.15))
pieces = {"train": frame.iloc[:train_end], "validation":
frame.iloc[train_end:validation_end],
          "test": frame.iloc[validation_end:]}

model = DecisionTreeClassifier(criterion="gini", max_depth=None, min_samples_leaf=1,
                               min_samples_split=2, random_state=42)
model.fit(train_df[FEATURES], train_df["label"])
```

Performance

Accuracy, precision, recall and F1-score

Table 11. Test-set metrics per class

Class	Precision	Recall	F1-score	Support
Horizontal	0.9905	0.9968	0.9937	315
Static	0.9935	1.0000	0.9967	306
Vertical	0.9968	0.9844	0.9906	321
macro avg	0.9936	0.9937	0.9937	942
weighted avg	0.9937	0.9936	0.9936	942

Table 12. Validation-set metrics per class

Class	Precision	Recall	F1-score	Support
Horizontal	0.9968	0.9968	0.9968	315
Static	0.9967	1.0000	0.9984	306
Vertical	0.9969	0.9938	0.9953	320
macro avg	0.9968	0.9969	0.9968	941
weighted avg	0.9968	0.9968	0.9968	941

The overall test accuracy is **99.36 %** (validation 99.68 %), with a macro-averaged F1-score of 0.9937. Static is never missed (recall 1.00), and the lowest recall is that of Vertical (0.984).

Train / validation / test accuracy

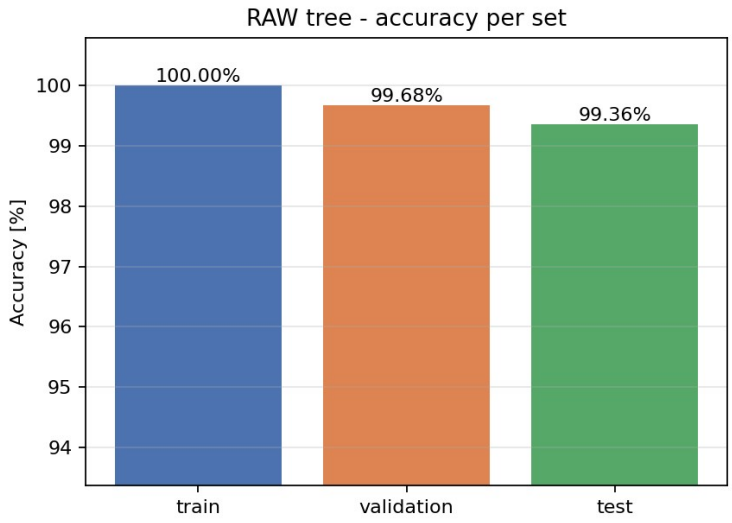


Figure 9. Accuracy of the RAW tree on the three sets.

Table 13. Accuracy comparison

Set	Samples	Accuracy
Train	4390	100.00 %
Validation	941	99.68 %
Test	942	99.36 %
Generalisation gap (train – test)		0.64 percentage points

The tree classifies the training set perfectly and loses 0.64 points on the test set. Because it was allowed to grow until every leaf was pure, it fits some noise of the training block, but the loss is small. The three sets come from different time blocks of the recordings, so this gap is a more honest estimate than one from a random split. It still does not measure how the model would work on a new recording session.

Confusion matrix and misclassifications

Table 14. Test confusion matrix (rows = true class, columns = predicted class)

True \ Predicted	Horizontal	Static	Vertical
Horizontal	314	0	1
Static	0	306	0
Vertical	3	2	316

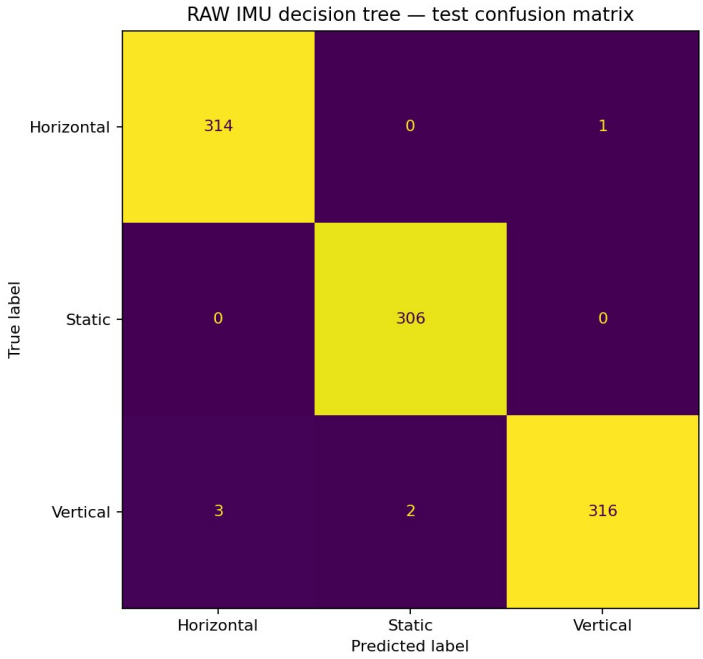


Figure 10. Test confusion matrix (heatmap of the counts in Table 14).

Only 6 of the 942 test samples are misclassified: 1 Horizontal sample as Vertical, 3 Vertical samples as Horizontal and 2 Vertical samples as Static. No Static sample is misclassified, but two Vertical samples are labelled Static, which lowers the precision of Static to 0.994.

Table 15. The misclassified test samples (acceleration in mg, angular rate in dps)

True	Predicted	Row	ax	ay	az	gx	gy	gz
Horizontal	Vertical	1940	63	88	837	33	-18	40
Vertical	Static	1816	-44	5	-575	-1	-61	14
Vertical	Horizontal	1820	188	33	844	102	32	5
Vertical	Static	1842	-7	22	726	-61	1	-24
Vertical	Horizontal	1858	195	56	1337	115	84	13
Vertical	Horizontal	1859	199	87	1406	110	84	17

All six errors fall on the moments where a single sample looks like another class:

- **Two Vertical samples predicted as Static (rows 1816, 1842).** Both have almost no horizontal acceleration (|ax| < 45 mg, ay 5 and 22 mg) and an angular rate in gyro_x of about 1 dps, which is what a resting board looks like. In the training block, 1,419 samples with |ax| < 50 mg, ay between 0 and 25 mg and |gyro_x| < 2 dps are Static and only 16 are Vertical, so the tree labels this region Static. These samples come from turning points of the up-and-down movement.
- **Three Vertical samples predicted as Horizontal (rows 1820, 1858, 1859).** They have the largest angular rates of the Vertical test block (gyro_x between 102 and 115 dps). This combination is rare: only 12 training samples have gyro_x above 90 dps together with ax between 150 and 250 mg, all of them Vertical, so the thresholds the tree places there are based on very few samples and extend badly to the test block.
- **One Horizontal sample predicted as Vertical (row 1940).** Its horizontal acceleration is momentarily small (ax 63 mg), as at the reversal point of the sideways movement, while the rotation is moderate.

A single sample cannot show a repeating pattern, so samples taken at a turning point or at a peak can be confused. This is the main limit of the RAW model and the reason for the windowed features of Task 3.

Feature importance

Table 16. Gini importance of the raw features

Rank	Feature	Importance	Share
1	ACC_Y	0.4610	46.1%
2	ACC_X	0.3173	31.7%
3	GYR_Z	0.1977	19.8%
4	ACC_Z	0.0106	1.1%
5	GYR_X	0.0078	0.8%
6	GYR_Y	0.0056	0.6%

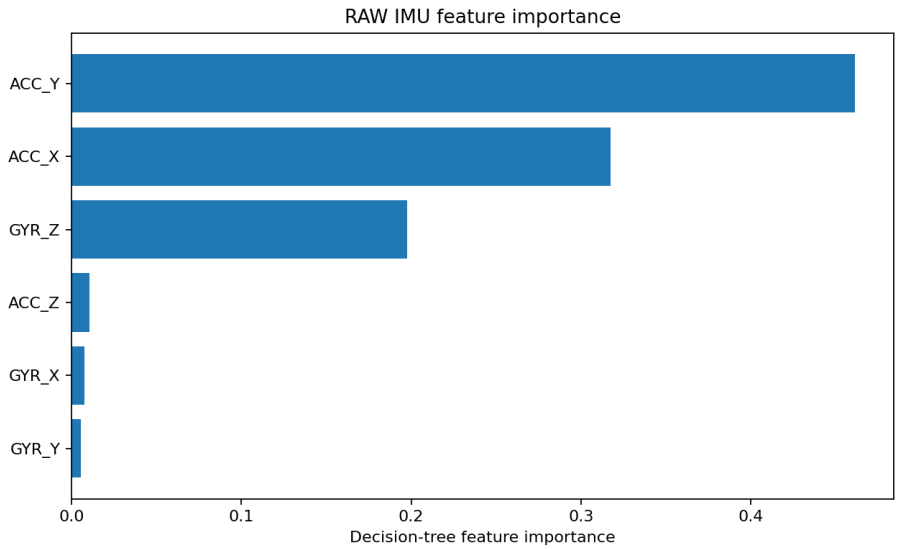


Figure 11. Feature importance ranking of the RAW tree.

Acc_Y (46 %), Acc_X (32 %) and Gyro_Z (20 %) account for 98 % of the importance. This agrees with the exploratory analysis of Task 1: the Horizontal movement is mostly along the x axis with a strong rotation about z, and the small, steady acc_y values (about 13 mg when resting) are what separates Static from the moving classes. Acc_Z, Gyro_X and Gyro_Y are used little. Note that the importance of Acc_Y comes largely from splits at differences of a few mg (for example thresholds at 19.5, 8.5 and −8.5 mg), which are at the level of sensor noise and make the tree more specific to the recording than necessary.

Tree structure and statistics

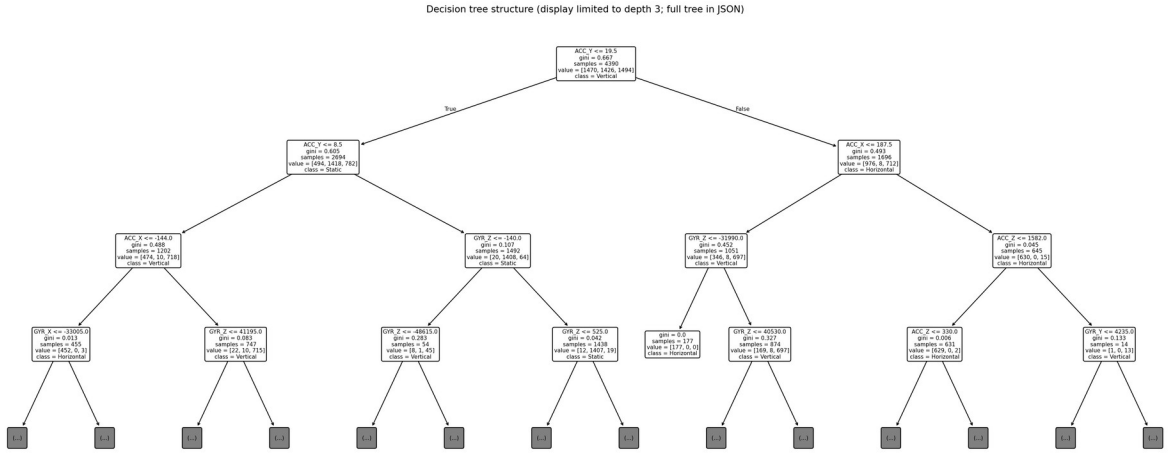


Figure 12. The first three levels of the RAW tree (the full tree is in the JSON file).

Table 17. Nodes and leaves at each depth

Depth	Nodes	Leaves
0	1	0
1	2	0
2	4	0
3	8	1
4	14	9
5	10	4
6	12	4
7	16	9
8	14	9
9	10	8
10	4	4

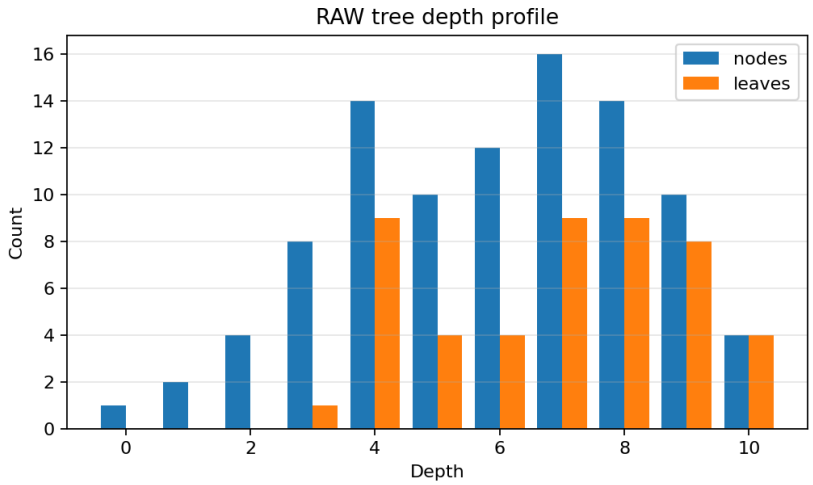


Figure 13. Depth profile of the tree.

Table 18. Tree statistics

Metric	Value
Depth	10
Number of leaves	48
Number of nodes	95
Serialised model size (decision_tree_raw.json)	43.68 KB
Generated C code (source + header)	4954 bytes of text
Estimated size of the tree table in C (9 bytes per node)	855 bytes
scikit-learn predict() of one row (reported by the script)	488 µs
Walking the exported tree, host PC (one sample)	1.671 µs
Estimated worst case on the MCU (depth × 12 cycles, 84 MHz)	1.429 µs (120 cycles)

The tree has depth 10, 48 leaves and a 43.7 KB JSON file. **About the latency figures:** the 488 µs reported by the script is the time of one scikit-learn predict() call and consists mostly of library and pandas overhead, so it says little about the cost of the tree. Walking the exported tree takes about 1.7 µs on a PC, and a C implementation on the 84 MHz microcontroller is estimated at roughly 1.4 µs in the worst case (about 12 cycles per level). These are estimates; the real time has to be measured on the board. The RAW model needs no buffer and no preparation of the input, and it can decide at every sample.

Because the tree was allowed to grow without limit, it is larger than needed. An extra run with max_depth 6 and min_samples_leaf 5 gave the same test accuracy (99.36 %) with 25 leaves and a 23.3 KB JSON file, about half the size. The results in this report use the default settings above, which are kept for all tasks.

Saved model (JSON) and C export

The complete tree is saved in decision_tree_raw.json (43.7 KB). The file lists the six feature names and the classes, and for every node: its id, the feature index and name and the threshold of a split (the left child is taken when feature ≤ threshold), the ids of both children, whether the node is a leaf, the

predicted class, the class counts, the number of samples and the impurity. The same script generates decision_tree_raw.c and decision_tree_raw.h for the board, which take the six values in the order Acc_X, Acc_Y, Acc_Z, Gyro_X, Gyro_Y, Gyro_Z in mg and mdps. The generated C code was compiled and compared with the Python model on all 6,273 logged samples and gave the same class for every one of them.

```
{ "format": "imu_decision_tree_v1",
  "features": ["ACC_X", "ACC_Y", "ACC_Z", "GYR_X", "GYR_Y", "GYR_Z"],
  "classes": ["Horizontal", "Static", "Vertical"],
  "node_count": 95, "tree_depth": 10, "leaf_count": 48,
  "nodes": [ { "node_id": 0, "feature_name": "ACC_Y", "threshold": 19.5,
               "left_child": 1, "right_child": ..., "is_leaf": false, ... }, ... ] }
```

Summary and limitations

- A depth-10 tree with 48 leaves classifies single samples with 99.36 % test accuracy from the six raw IMU values.
- The only errors are between Vertical and the other two classes, at turning points and peaks of the motion where one sample is ambiguous.
- The model needs no preparation of the input and a few microseconds per decision (estimated), but it is larger than necessary (unlimited depth).
- Limitations: about 29 s of data per class from one recording of one person, chronological blocks of the same recordings (not independent sessions), and untrimmed recordings. The accuracy therefore applies to these recordings, and a test on separately recorded sessions would be a stronger evaluation.
- This model is the baseline for Task 3 (windowed features), Task 4 (quaternion) and Task 5 (quaternion with feature selection).

Task 3: RAW+FE Model – 6-Axis with Feature Engineering

Windowing

Instead of classifying single samples (Task 2), the RAW+FE model classifies short **windows** of the six raw signals using statistics computed over each window. A window should be long enough to contain a full motion cycle, so that its statistics do not depend on the instant at which it starts.

Table 19. Window configuration

Parameter	Value	Comment
Window size	42 samples	About 0.59 s at the effective sampling rate of 71.3 Hz of the logs (0.42 s at 100 Hz). The Horizontal and Vertical movements repeat at about 2 Hz (0.5 s per cycle), so a window holds a little more than one cycle.
Stride (hop)	21 samples	Consecutive windows share half of their samples.
Overlap	50 %	Uses the data efficiently and avoids missing a movement at a window edge.
Windows are cut	inside each train / validation / test segment	No window crosses a segment boundary, so no sample is shared between the sets.
Window label	class of the recording	Each recording contains one activity.
Windows obtained	287 (205 / 41 / 41)	Train / validation / test.

The windows are defined by a number of samples because the generated C function takes a window of a fixed number of samples. The logged timestamps are slightly irregular (steps of 10 and 20 ms), so the duration of a window varies a little around the value above.

Feature extraction

For each of the six axes the six statistics of the assignment are computed over the window:

Table 20. Statistical features

Statistic	Definition
mean	Average of the window.
var	Variance (population variance, divided by n).
max	Largest value.
min	Smallest value.
p2p	Peak-to-peak: max − min.
energy	Mean of the squared values (sum of squares divided by n), which does not depend on the number of samples in the window.

Total number of features: 36 (6 axes × 6 statistics), named for example ACC_X__mean or GYR_Z__energy. Acceleration is in mg and angular rate in mdps, as logged. All 36 features are given to the tree: **no feature selection** is applied in this task (selection is the subject of Task 5).

Feature extraction code

```
STATISTICS = ["mean", "var", "max", "min", "p2p", "energy"]

def extract_window_features(window):          # window: (n_samples, 6) raw values
    x = np.asarray(window, dtype=np.float64)
    mean, var = x.mean(axis=0), x.var(axis=0)          # var: divide by n
    mx, mn = x.max(axis=0), x.min(axis=0)
    energy = (x * x).mean(axis=0)                      # mean of the squares
```

```
    out = {}
    for i, axis in enumerate(RAW_CHANNELS):
        for stat, v in zip(STATISTICS, (mean[i], var[i], mx[i], mn[i], mx[i] - mn[i],
energy[i])):
            out[f"{axis}__{stat}"] = float(v)
    return out

for start in range(0, len(segment) - WINDOW + 1, STRIDE):     # windows inside one segment
    features = extract_window_features(raw[start:start + WINDOW])
```

Model and data

The decision tree uses exactly the hyperparameters of Task 2 (criterion gini, max_depth None, min_samples_leaf 1, min_samples_split 2, random_state 42); the defaults of both scripts are identical. The data split is the chronological split of Task 2: the first 70 % of each recording for training, the next 15 % for validation and the last 15 % for testing, now cut into windows.

Table 21. Windows per class in each set

Class	Train	Validation	Test	Total
Horizontal	69	14	14	97
Static	66	13	13	92
Vertical	70	14	14	98
All	205	41	41	287

Caveat. There are only 205 / 41 / 41 windows, so a single error in the test set changes the accuracy by about 2.4 points. The windows overlap by 50 % inside a set, but because the sets are different time blocks, a test window never shares a sample with a training window.

Performance

Table 22. Test-set metrics per class

Class	Precision	Recall	F1-score	Support
Horizontal	1.0000	1.0000	1.0000	14
Static	1.0000	1.0000	1.0000	13
Vertical	1.0000	1.0000	1.0000	14
macro avg	1.0000	1.0000	1.0000	41
weighted avg	1.0000	1.0000	1.0000	41

Table 23. Validation-set metrics per class

Class	Precision	Recall	F1-score	Support
Horizontal	1.0000	1.0000	1.0000	14
Static	1.0000	1.0000	1.0000	13
Vertical	1.0000	1.0000	1.0000	14
macro avg	1.0000	1.0000	1.0000	41
weighted avg	1.0000	1.0000	1.0000	41

Table 24. Accuracy progression, compared with Task 2

Set	RAW+FE (windows)	RAW (samples, Task 2)
Train	100.00 % (205 windows)	100.00 % (4390 samples)
Validation	100.00 % (41 windows)	99.68 % (941 samples)
Test	100.00 % (41 windows)	99.36 % (942 samples)
Gap (train – test)	0.00 points	0.64 points

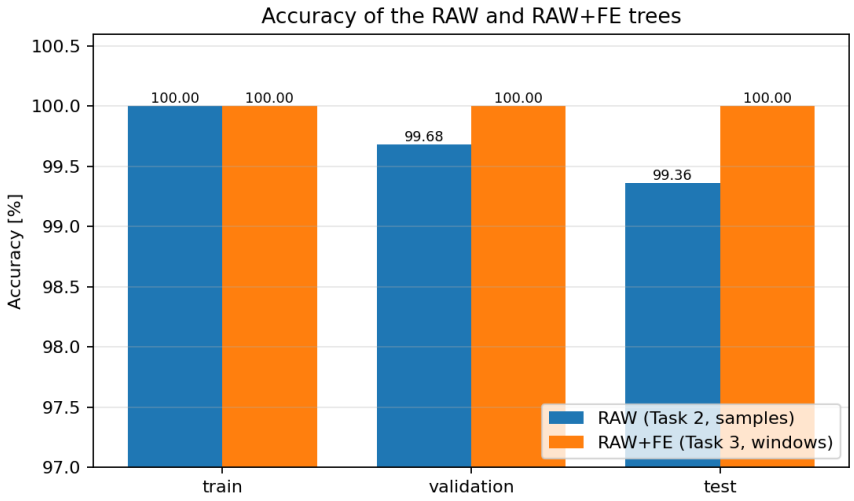


Figure 14. Train, validation and test accuracy of the RAW+FE and RAW trees.

The RAW+FE tree classifies **all train, validation and test windows correctly** (macro-F1 = 1.00). The RAW tree of Task 2 reached 99.36 % on the test set, with 6 errors in 942 samples.

What this does and does not show. The test set has 41 windows. A perfect score on 41 windows is statistically consistent with a true accuracy as low as about 93 % (one-sided 95 % bound), while the RAW model was tested on 942 samples. The two numbers are measured on different units (windows and samples) and are not directly comparable. What can be said is that windowing removes the individual-sample errors of the RAW model (Table 15), and that the windowed classes are very well separated.

Confusion matrix and per-class analysis

Table 25. Test confusion matrix (rows = true class, columns = predicted class)

True \ Predicted	Horizontal	Static	Vertical
Horizontal	14	0	0
Static	0	13	0
Vertical	0	0	14

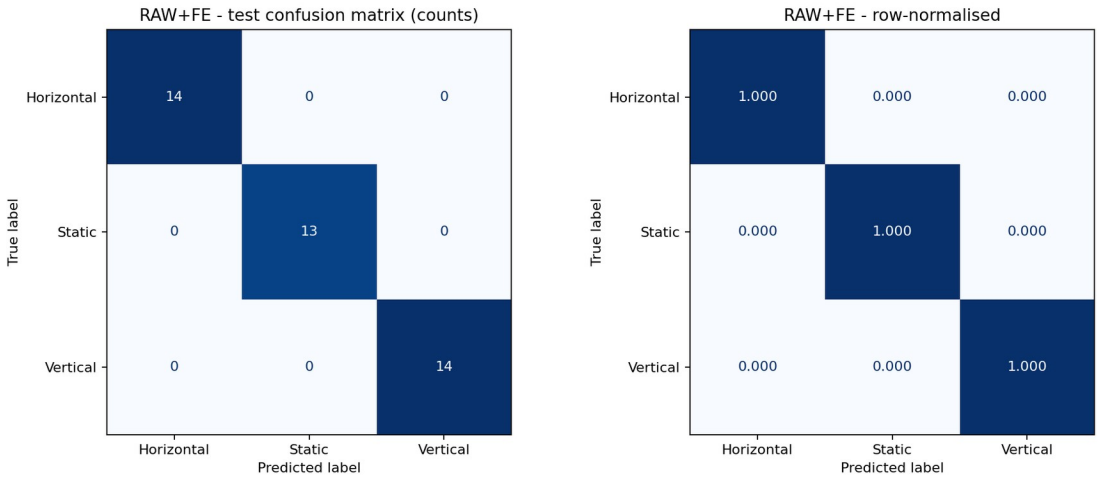


Figure 15. Test confusion matrix of the RAW+FE model as a heatmap (counts and row-normalised).

All 41 test windows are on the diagonal, so precision, recall and F1-score are 1.00 for each class. In Task 2, the Vertical class was the weakest (recall 0.984) because single samples at a peak or turning point resembled the other classes. A window contains the whole cycle, so these moments are averaged out. In the windows the classes are also far apart in the statistics: for example the peak-to-peak of acc_x is above 1,128 mg for Horizontal windows and below it for Static and Vertical windows.

Feature importance

Table 26. The most important engineered features (Gini importance)

Rank	Feature	Importance	Share
1	ACC_X__p2p	0.5027	50.3%
2	ACC_X__energy	0.4973	49.7%
3	ACC_X__mean	0.0000	0.0%
4	ACC_X__var	0.0000	0.0%
5	ACC_X__min	0.0000	0.0%
6	ACC_X__max	0.0000	0.0%
7	ACC_Y__mean	0.0000	0.0%
8	ACC_Y__var	0.0000	0.0%

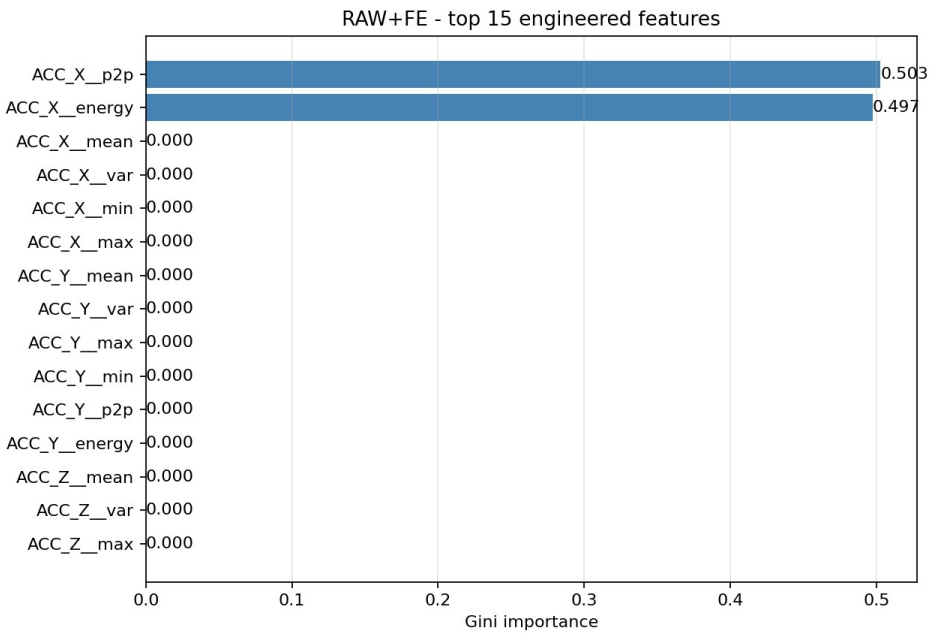


Figure 16. Importance of the top 15 engineered features (all others are 0).

The tree uses only **two of the 36 features, both from the x axis of the accelerometer**: ACC_X__p2p (50.3 %) and ACC_X__energy (49.7 %). It consists of two thresholds:

- ACC_X__p2p > 1128 mg → Horizontal (the sideways movement gives a large acc_x swing).
- Otherwise ACC_X__energy ≤ 1707 → Static; else → Vertical.

How to read this ranking. An importance of 0 does not mean a feature is useless. When several features separate the classes perfectly, the tree uses the first one it finds and ignores the rest. A supplementary check shows this: a depth-2 tree built on a **single** feature reaches at least 95 % accuracy on the validation and test windows for **25 of the 36 features** (for example ACC_X__max, ACC_Y__var, ACC_X__energy). The engineered features are therefore highly redundant, and with this dataset the ranking shows which features the tree picked, not which ones are the only informative ones. By statistic, the spread measures (peak-to-peak and energy) carry all the importance of this tree, and the mean, variance, maximum and minimum carry none.

Complexity compared with Task 2

Table 27. RAW versus RAW+FE

Metric	RAW (Task 2)	RAW+FE (Task 3)
Input to the tree	1 sample	42-sample window
Features	6	36
Tree depth	10	2
Leaves	48	3
Nodes	95	5
JSON size [KB]	43.679	4.837
Estimated size in C [bytes]	855	45

Metric	RAW (Task 2)	RAW+FE (Task 3)
Train accuracy [%]	100.0	100.0
Validation accuracy [%]	99.68	100.0
Test accuracy [%]	99.36	100.0
Generalisation gap [points]	0.64	0.0
Window buffer [bytes]	0	1008
Work before the tree	none	36 statistics per window
Decision rate	every sample	every 21 samples
Delay to first decision	1 sample	42 samples

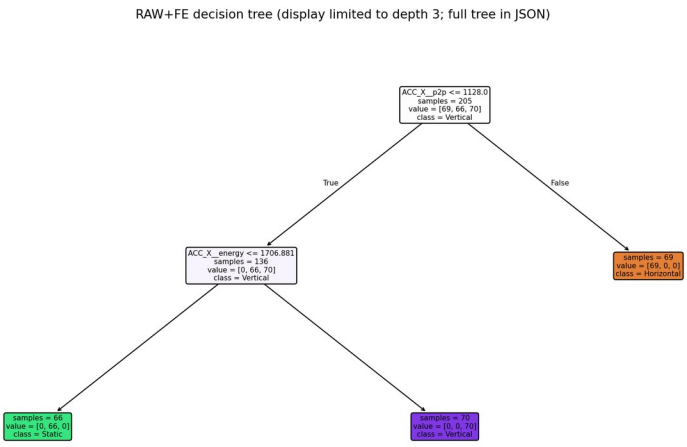


Figure 17. The RAW+FE decision tree (3 leaves).

The tree itself is much smaller. Depth falls from 10 to 2, the leaves from 48 to 3, the JSON file from 43.7 KB to 4.8 KB, and the estimated tree table in C from about 855 to 45 bytes. The tree needs two comparisons per decision. The estimated worst-case time of the tree on the 84 MHz microcontroller falls from about 1.4 µs to 0.3 µs.

The total system is more expensive. The tree needs its input prepared:

- RAM: a buffer for one window of 42 samples × 6 axes × 4 bytes = 1008 bytes. RAW needs none.
- Computation: 36 statistics must be calculated for every window (about 40 µs per window in Python on a PC, which says little about the microcontroller). Only two of them, ACC_X__p2p and ACC_X__energy, are actually used by the tree, so a deployed version could compute just those two. In the professor's board firmware, the feature step measured about 880 µs against 1.89 µs for the tree (for its own feature set), so feature extraction costs far more than the tree.
- Delay: the first decision comes after 42 samples (about 0.59 s here), then one decision every 21 samples (about 0.29 s). RAW decides at every sample.

The choice between the models is therefore a trade of memory, delay and feature computation against smoothing: windowed features remove the single-sample errors and allow a tiny tree, but they need a buffer, a calculation step and some data before the first decision.

Check the professor's firmware numbers (880 µs features, 1.89 µs tree) against the video before submitting.

Saved model (JSON) and C export

The tree is saved in decision_tree_raw_fe.json (4.8 KB), with the same node format as the Task 2 file. It also stores the window size and stride, the 36 feature names, the statistic order and the definition of every statistic, so the model can be reproduced exactly. The script generates decision_tree_raw_fe.c and .h for the board: the function imu_feature_tree_predict_window() takes a window of 42 × 6 raw values (mg and mdps), calculates the 36 features and runs the tree. The generated C code was compiled and compared with the Python model on 881 windows taken at many offsets of the three recordings and gave the same class for every one.

```
{ "format": "imu_feature_decision_tree_v2",
  "raw_channels": ["ACC_X", "ACC_Y", "ACC_Z", "GYR_X", "GYR_Y", "GYR_Z"],
  "window_size_samples": 42, "recommended_stride_samples": 21,
  "statistic_order_per_axis": ["mean", "var", "max", "min", "p2p", "energy"],
  "all_engineered_features": ["ACC_X__mean", ..., "GYR_Z__energy"],          // 36
  "tree_depth": 2, "leaf_count": 3, "node_count": 5,
  "nodes": [ { "node_id": 0, "feature_name": "ACC_X__p2p", "threshold": 1128,
               "left_child": 1, "right_child": 4, "is_leaf": false, ... }, ... ] }
```

Summary and limitations

- 36 features (6 statistics on 6 axes) over 42-sample windows with 50 % overlap give a tree of depth 2 with 3 leaves and a 4.8 KB JSON file, and 100 % accuracy on the validation and test windows.
- Compared with Task 2, the tree is much smaller (depth 10 → 2, 48 → 3 leaves), but the system needs a 1008-byte window buffer, the feature calculation, and about 0.59 s before the first decision.
- The classes are so well separated that two features from one axis suffice, and 25 of the 36 features work on their own, so the importance ranking is not very informative.
- Limitations: 41 test windows, one recording of about 29 s per class from one person, chronological blocks of the same recordings (not independent sessions), and recordings that are not trimmed at the start and end. Several longer recordings per class would give much more reliable numbers.

Tasks 4 and 5: Data, protocol and the main caveat

Tasks 4 and 5 use **the same recordings, the same split, the same hyperparameters and the same window** as Tasks 2 and 3, so the four models can be compared directly:

- Data: the Task 1 recordings (Horizontal.csv, Static.csv, Vertical.csv), which contain the quaternion q0–q3 logged next to the accelerometer and gyroscope. The quaternion columns are used here, exactly as logged.
- Split: each recording is cut chronologically, 70 % train, 15 % validation, 15 % test (as in Task 2).
- Hyperparameters: the defaults of the scripts, identical in all four tasks (criterion gini, max_depth None, min_samples_leaf 1, min_samples_split 2, random_state 42).
- Windows (Task 5): 42 samples, stride 21 (50 % overlap), cut inside each segment, as in Task 3.

As an independent check, both models were also run on three quaternion-only logs (Quat_Horizontal.csv, Quat_Static.csv, Quat_Vertical.csv; about 30 s each at 100 Hz).

Main caveat: a quaternion describes the orientation of the board, not its motion. In every recording the board was held at a slightly different orientation, and without a magnetometer the heading is arbitrary. Moreover, the orientation drifts during a recording. A tree can therefore separate the classes by the orientation level, which works inside the training block and fails when the orientation moves. This is exactly what happens on the test block of the Task 1 recordings (Tasks 4 and 5), and it is why the quaternion models are much weaker than the IMU models in this assignment.

Table 28. Mean quaternion of the training and the test block (Task 1 recordings)

Recording	Block	q0	q1	q2	q3
Horizontal	train / test	0.003 / 0.015	0.061 / 0.067	0.986 / 0.991	−0.131 / −0.105
Static	train / test	0.006 / 0.006	0.000 / 0.000	0.997 / 0.997	−0.078 / −0.078
Vertical	train / test	0.081 / 0.074	0.133 / 0.115	0.986 / 0.990	+0.038 / −0.015

The Static quaternion does not move at all. The Vertical q3 crosses zero (+0.038 in the training block, −0.015 in the test block), and the Horizontal q3 also shifts. The trees below use thresholds at about this level.

Task 4 – QUAT model

Features and training

The four quaternion components of a single sample (q0 the scalar part, q1, q2, q3 the vector part) are the only features. Each sample is classified on its own, as in the RAW model, but from 4 values instead of 6. The script is model_quaternion.py; it was changed only to follow the assignment (file names, accuracy gap, latency and importance outputs).

```
FEATURES = ["q0", "q1", "q2", "q3"]
model = DecisionTreeClassifier(criterion="gini", max_depth=None, min_samples_leaf=1,
                               min_samples_split=2, random_state=42)     # same as Tasks 2-
3
model.fit(splits["train"][FEATURES], splits["train"]["label"])           # chronological
split
```

Accuracy, precision, recall and F1-score

Table 29. Test-set metrics per class

Class	Precision	Recall	F1-score	Support
Horizontal	0.5769	1.0000	0.7317	315
Static	1.0000	1.0000	1.0000	306
Vertical	1.0000	0.2804	0.4380	321
macro avg	0.8590	0.7601	0.7232	942

Table 30. Validation-set metrics per class

Class	Precision	Recall	F1-score	Support
Horizontal	1.0000	1.0000	1.0000	315
Static	1.0000	1.0000	1.0000	306
Vertical	1.0000	1.0000	1.0000	320
macro avg	1.0000	1.0000	1.0000	941

The model is perfect on the training and validation blocks but reaches only **75.48 % on the test block** (macro-F1 0.72). Static and Horizontal are always found; the recall of Vertical is only 0.28.

Generalisation gap

Table 31. Train / validation / test accuracy

Set	Samples	Accuracy
Train	4390	100.00 %
Validation	941	100.00 %
Test	942	75.48 %
Generalisation gap (train – test)		24.52 percentage points

A gap of **24.5 points** is a clear case of overfitting, in the sense that the tree learned something specific to the training block. The RAW model of Task 2 had a gap of 0.64 points on the same split. The next section shows what the tree learned.

Confusion matrix and analysis of the errors

Table 32. Test confusion matrix (rows = true class)

True \ Predicted	Horizontal	Static	Vertical
Horizontal	315	0	0
Static	0	306	0
Vertical	231	0	90

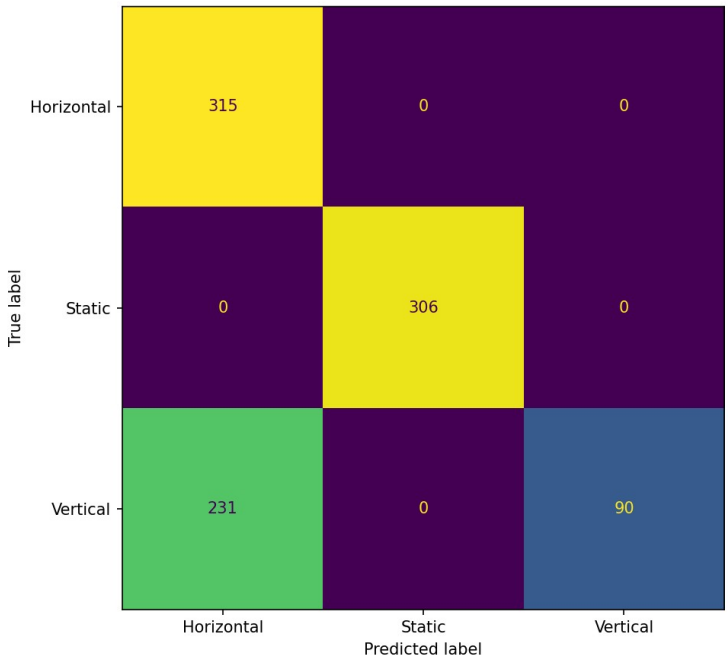


Figure 18. QUAT test confusion matrix (heatmap of Table 32).

All 231 errors are **Vertical samples classified as Horizontal** (231 of 321). The tree is:

- q3 > -0.0015 → Vertical.
- Otherwise (q3 ≤ -0.0015): q2 ≤ 0.9965 → Horizontal; q2 above it → Static if q3 ≤ -0.069, else Horizontal.

So Vertical is recognised as "q3 is slightly positive". In the training block this is true (mean q3 = +0.038), but in the test block the mean q3 of the Vertical recording has moved to −0.015 (Table 28). Most of its test samples now fall below the threshold and are labelled Horizontal. The tree has learned the orientation of the board in that part of the recording, not the up-and-down movement.

Quaternion component importance

Table 33. Gini importance of q0–q3

Rank	Component	Importance
1	q3	0.5739
2	q2	0.4261
3	q1	0.0000
4	q0	0.0000

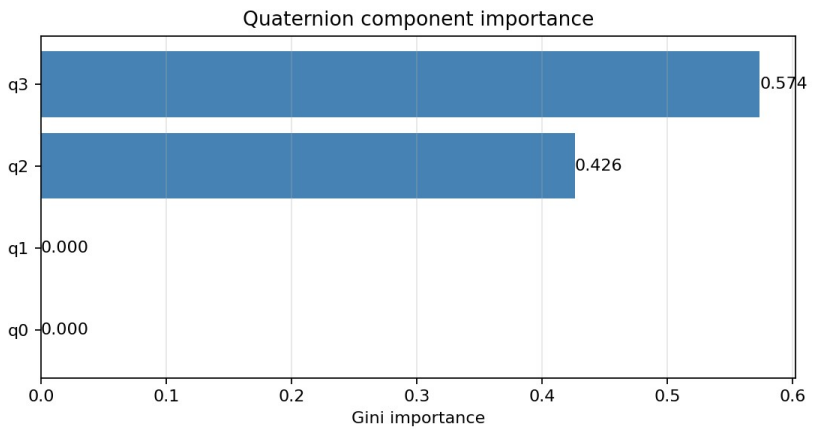


Figure 19. Importance of the quaternion components.

Only q3 (57 %) and q2 (43 %) are used; q0 and q1 have importance 0. The most discriminative component is q3, which carries the orientation difference between the classes (Table 28: −0.078 for Static, about −0.13 for Horizontal and about +0.04 for Vertical). q2 only helps to isolate the Static orientation.

Compactness

Table 34. Size of the QUAT tree compared with the RAW tree

Metric	RAW (Task 2)	QUAT (Task 4)
Features per sample	6	4
Depth	10	3
Leaves	48	4
Nodes	95	7
JSON size [KB]	43.7	3.2
Estimated tree size in C [bytes]	855	63
Estimated MCU worst case [µs, 84 MHz]	1.43	0.43
Test accuracy	99.36 %	75.48 %

The QUAT tree is much smaller than the RAW tree (7 nodes against 95, 3.2 KB against 43.7 KB). **Advantages of the quaternion representation:** four features instead of six, a very small tree, and the orientation is already computed by the sensor-fusion block when that block is running for other reasons. **Costs:** the quaternion has to be computed first. In the professor's firmware log the fusion step (MotionFX) takes about 1,173 µs on the NUCLEO board, against about 1.9 µs for the tree, so the fusion, not the tree, dominates the cost. And, as shown above, the small tree is small because it learned a shortcut.

Overfitting characteristics: robustness checks

Two checks show how fragile the quaternion model is (Figure 20 and Table 35):

- **Leave-one-block-out.** Each recording was cut into 5 consecutive blocks and each block was held out in turn. The mean accuracy over the 5 folds is 94.18 %, but the worst fold is only 81.59 % (the last block of each recording: all 231 errors are Vertical samples). The first block is wrong too, with 134 Horizontal errors.
- **Transfer between datasets.** Trained on the Task 1 recordings and tested on the quaternion-only logs, the model scores 34.4 %, and 33.5 % in the other direction; it predicts almost a single class for every sample. The two datasets use different orientations (and possibly a different quaternion convention), so the thresholds do not carry over.

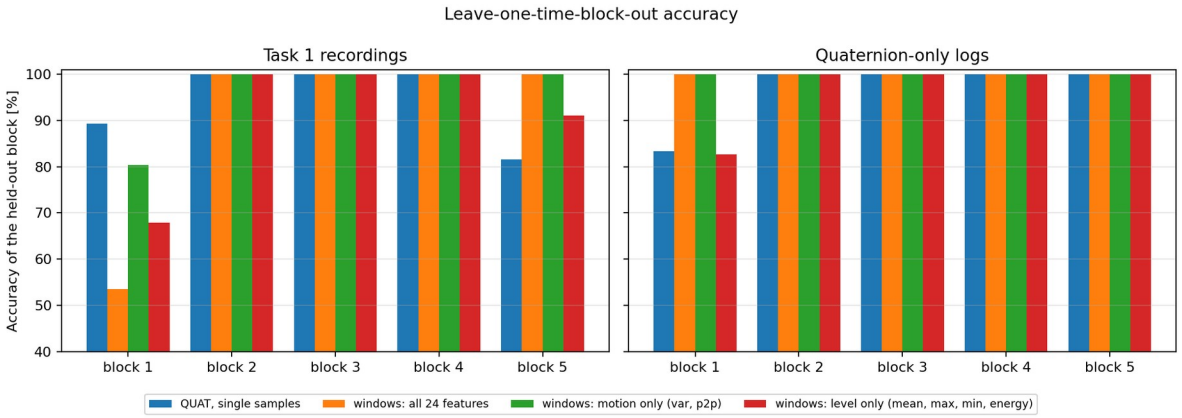


Figure 20. Leave-one-time-block-out accuracy for single quaternion samples and for window features of different kinds (see the motion-versus-level analysis in Task 5).

Table 35. The QUAT model on the second dataset (quaternion-only logs)

Check	Result
Chronological split (same protocol)	train 100.00 %, validation 100.00 %, test 100.00 % (depth 6, 7 leaves)
Leave-one-block-out, mean / worst fold	96.66 % / 83.29 %
Transfer, Task 1 recordings → logs / logs → Task 1 recordings	34.4 % / 33.5 %

On the logs, the test block happens to lie in the same orientation range as the training block, so the chronological test reaches 100 %, but the first block of the Vertical log (which starts near the Static orientation) is classified wrongly in the leave-one-block-out check, and the transfer fails. One recording per class from a single session cannot tell a quaternion model that works from one that has memorised the session.

Insight: effectiveness of the compact 4-feature representation

The 4-feature quaternion tree is the smallest model per sample, but on the shared test split it is the least accurate model of the whole assignment (75.5 %, against 99.4 % for the 6-feature RAW tree), and it does not transfer to another session. A quaternion alone says where the board points. Because each activity was recorded at its own orientation, that information is enough inside a recording and useless outside it. For recognising motion in any orientation, the quaternion values are not a reliable input by themselves.

Task 5 – QUAT+FE model

Windows and features

The windows are those of Task 3: 42 samples (about 0.59 s at the effective 71 Hz of the logs), stride 21 (50 % overlap), cut inside each train / validation / test segment. This gives 205 / 41 / 41 windows (train / validation / test).

Features: the six statistics of Task 3 (mean, variance, maximum, minimum, peak-to-peak and energy, the mean of the squares) for each quaternion component: **4 × 6 = 24 candidate features** (q0__mean … q3__energy).

```
CHANNELS = ["q0", "q1", "q2", "q3"];   STATS = ["mean", "var", "max", "min", "p2p",
"energy"]
def extract_features(w):                        # w: (42, 4) quaternion window
    x = np.asarray(w, float)
    mean, var, mx, mn, energy = x.mean(0), x.var(0), x.max(0), x.min(0), (x * x).mean(0)
    return {f"{ch}__{s}": float(v) for i, ch in enumerate(CHANNELS)
            for s, v in zip(STATS, (mean[i], var[i], mx[i], mn[i], mx[i] - mn[i],
energy[i]))}
```

Feature selection (as in MEMS Studio)

The same two ideas as in MEMS Studio's Automatic Filter & Feature Selection were implemented in Python and applied to the training windows only, so that the validation and test windows do not influence the selection:

- **ANOVA F-statistic** of each candidate feature: how much the class means of a feature differ compared with its spread inside the classes. It is computed for all 24 features (Table 36).
- **Recursive Feature Elimination (RFE)** with a random-forest estimator: the forest is trained, the least important feature is removed, and this is repeated until one feature is left. The removal order ranks all 24 features, and the top K = 8 (one third) are kept.

Why a random forest. RFE with a single decision tree was tested first and rejected: one feature already separates the classes in the training windows, so the tree gives importance 0 to 22 of the 24 features, and RFE then removes the features in plain column order (q0__mean, q0__var, q0__max, …). Its ranking would carry no information.

Table 36. ANOVA F-statistics of all 24 quaternion features (training windows) and the RFE outcome

#	Feature	F	p-value	RFE rank	Selected
1	q3__p2p	2959	<0.001	3	yes
2	q3__var	2020	<0.001	1	yes
3	q1__max	936	<0.001	4	yes
4	q0__p2p	862	<0.001	13	no
5	q1__energy	693	<0.001	5	yes
6	q3__min	665	<0.001	2	yes
7	q1__mean	622	<0.001	16	no
8	q3__mean	517	<0.001	6	yes
9	q1__p2p	510	<0.001	14	no

#	Feature	F	p-value	RFE rank	Selected
10	q1__min	418	<0.001	18	no
11	q3__max	415	<0.001	7	yes
12	q0__var	403	<0.001	11	no
13	q2__p2p	335	<0.001	12	no
14	q0__max	328	<0.001	20	no
15	q0__energy	233	<0.001	19	no
16	q0__mean	159	<0.001	24	no
17	q2__min	143	<0.001	10	no
18	q2__var	129	<0.001	9	no
19	q1__var	101	<0.001	8	yes
20	q3__energy	99	<0.001	21	no
21	q2__energy	95	<0.001	15	no
22	q2__mean	94	<0.001	17	no
23	q0__min	77	<0.001	22	no
24	q2__max	48	<0.001	23	no

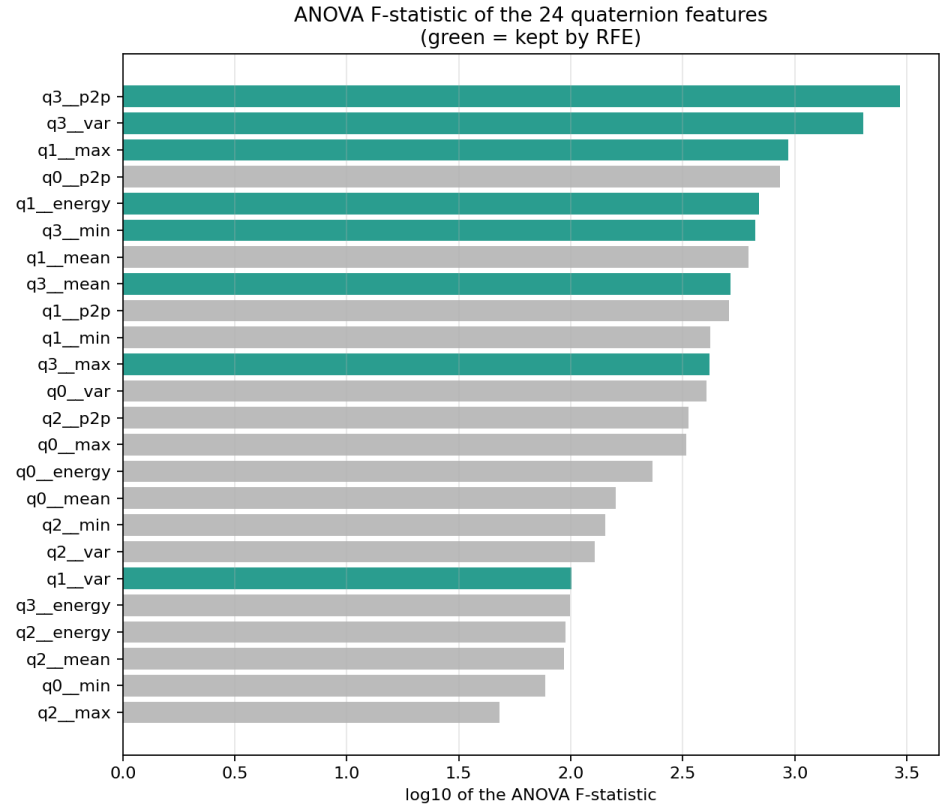


Figure 21. ANOVA F-statistics (log scale); green bars were kept by RFE.

Result: 24 candidate features → 8 selected: q3__var, q3__min, q3__p2p, q1__max, q1__energy, q3__mean, q3__max, q1__var. Every feature has $p < 0.001$, so all of them differ significantly between the classes.

Which features were eliminated, and why

- **All twelve q0 and q2 features.** RFE judges the features together, and the forest preferred the q3 and q1 statistics. This includes features with a high F-statistic on their own (q0__p2p, F = 862), whose information the selected features already cover well enough for the forest. The explanation is not simple redundancy: q0__p2p is almost uncorrelated with q3__p2p (r = 0.13).
- **Also eliminated among the q1 and q3 statistics:** q1__p2p, q1__min, q1__mean and q3__energy. The kept features are q3__var, q3__min, q3__p2p, q3__mean, q3__max, q1__max, q1__energy and q1__var.
- **Redundancy is not fully removed.** q3__var and q3__p2p are strongly correlated (r = 0.98) and so are q3__mean and q3__max (r = 0.99), and both pairs were kept. This is why the selection of 8 features is more generous than the final tree needs.

Note that the selection is made on the training block, where the classes are separated by the orientation level (q3__mean, q3__max, q3__min) as well as by the motion (q3__var, q3__p2p). RFE cannot know which of these will still be valid in the test block.

Training and performance

A decision tree with the Task 2 hyperparameters was trained on the 8 selected features.

Table 37. Test-set metrics per class

Class	Precision	Recall	F1-score	Support
Horizontal	0.6364	1.0000	0.7778	14
Static	1.0000	1.0000	1.0000	13
Vertical	1.0000	0.4286	0.6000	14
macro avg	0.8788	0.8095	0.7926	41

Table 38. Validation-set metrics per class

Class	Precision	Recall	F1-score	Support
Horizontal	1.0000	1.0000	1.0000	14
Static	1.0000	1.0000	1.0000	13
Vertical	1.0000	1.0000	1.0000	14
macro avg	1.0000	1.0000	1.0000	41

Table 39. Accuracy progression

Set	Windows	Accuracy
Train	205	100.00 %
Validation	41	100.00 %
Test	41	80.49 %
Generalisation gap (train – test)		19.51 percentage points

Table 40. QUAT+FE test confusion matrix (rows = true class)

True \ Predicted	Horizontal	Static	Vertical
Horizontal	14	0	0
Static	0	13	0
Vertical	8	0	6

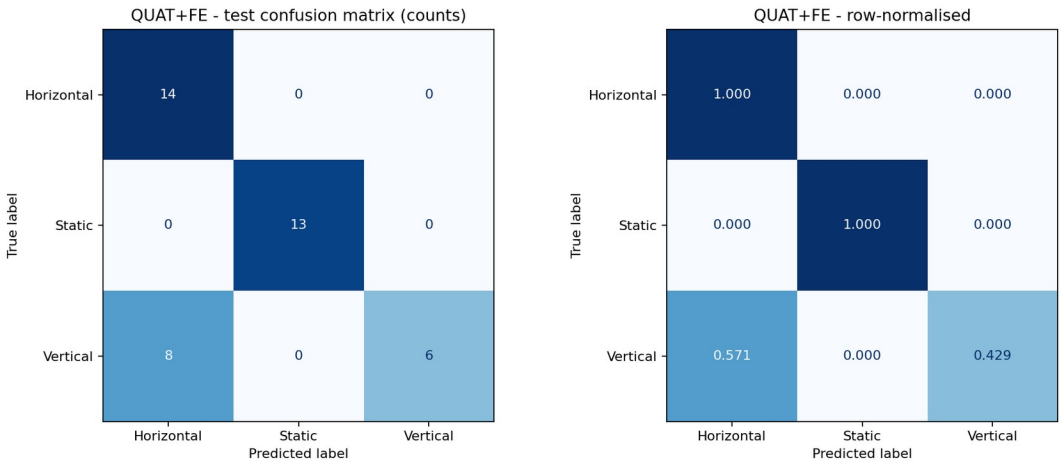


Figure 22. QUAT+FE test confusion matrix.

The model is perfect on train and validation but reaches **80.49 % on the test windows** (33 of 41), with a gap of 19.5 points. All 8 errors are Vertical windows classified as Horizontal (recall of Vertical 0.43), the same failure as in Task 4. With 41 windows each error costs 2.4 points, so the exact percentage is uncertain, but the pattern is clear.

Analysis of the selected features

Table 41. Importance of the features the tree uses

Feature	Gini importance
q3__max	0.5062
q1__energy	0.4938

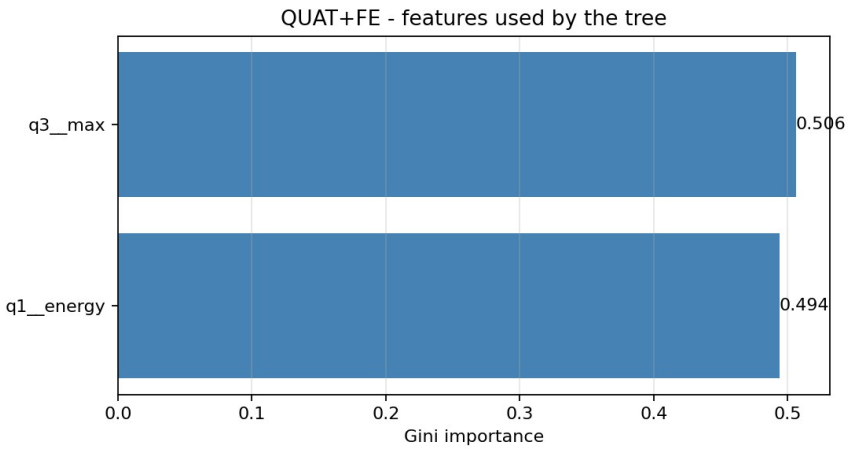


Figure 23. Importance of the features used by the QUAT+FE tree.

The tree uses **two features: q3__max and q1__energy**. Both are level statistics, not motion statistics:

- q3__max > 0.0025 → Vertical.
- Otherwise q1__energy ≤ 4.67e-5 → Static, else → Horizontal.

The motion features that RFE ranked first (q3__var, q3__p2p) are among the 8 inputs, but the root split of the tree is on the level feature q3__max, which separates Vertical from the other classes in the training windows. The tree therefore inherits the weakness of Task 4. The q3 maximum of the Vertical windows has a median of 0.049 in the training block, 0.026 in the validation block and −0.008 in the test block, so most Vertical test windows fall below the threshold of 0.0025 and are labelled Horizontal.

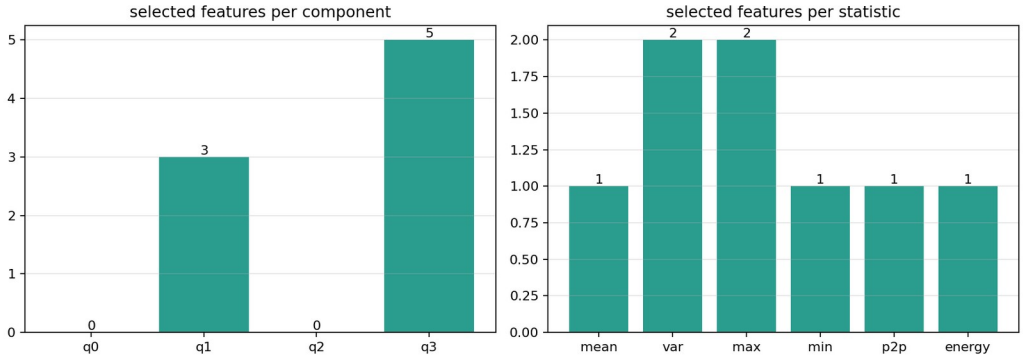


Figure 24. Selected features per quaternion component (left) and per statistic (right).

Component and statistic distribution. By component, q3 dominates (5 of the 8 selected features) followed by q1 (3); q0 and q2 contribute none. By statistic, var and max have 2 each and mean, min, p2p and energy 1 each. The q3 component, which carries the orientation difference between the classes (Table 28), is the one that dominates, in line with Task 4.

Complexity compared with Task 3

Table 42. QUAT+FE compared with RAW+FE

Metric	RAW+FE (Task 3)	QUAT+FE (Task 5)
Candidate -> selected features	36 -> 36	24 -> 8
Tree depth	2	2
Leaves	3	3
Nodes	5	5
JSON size [KB]	4.837	4.675
Estimated size in C [bytes]	45	45
Test accuracy [%]	100.0	80.49
Window buffer [bytes]	1008	672
Channels buffered	6	4

Both trees have the same shape (depth 2, 3 leaves, 5 nodes). The RAM for the window buffer is lower for the quaternion model, because only four channels are buffered (42 × 4 × 4 = 672 bytes against 1,008 bytes for six IMU channels). Because the final tree uses only q3__max and q1__energy, only q3 and q1 would have to be buffered. As for Task 4, the quaternion itself comes from the sensor-fusion step, which costs far more time than the feature calculation or the tree. The work before the tree is: sensor fusion, then the statistics of the selected features, then the tree.

Robustness: motion features against level features

To test the explanation above, the same window models were trained with three groups of features (default hyperparameters, no feature selection):

Table 43. Window models with different feature groups (Task 1 recordings, 41 test windows)

Feature group	Test accuracy, chronological split	Leave-one-block-out, mean	Leave-one-block-out, worst fold
all 24 features	80.49 %	90.71 %	53.57 %
motion only (var, p2p)	100.00 %	96.07 %	80.36 %
level only (mean, max, min, energy)	87.80 %	91.79 %	67.86 %

Table 44. The same on the quaternion-only logs, and transfer between the two datasets

Feature group	Test accuracy (logs, 60 windows)	Transfer Task 1 → logs	Transfer logs → Task 1
all 24 features	100.00 %	33.33 %	66.10 %
motion only (var, p2p)	98.33 %	98.37 %	66.10 %

Using only the **motion statistics (variance and peak-to-peak)** the window model reaches **100 % on the test windows of the Task 1 recordings** (against 80.5 % with all 24 features and 87.8 % with the level features only), and the worst leave-one-block-out fold improves from 53.6 % to 80.4 %. Trained on the

Task 1 recordings and tested on the quaternion logs, the motion-only model scores **98.4 %**, while the all-feature model and the single-sample model fail (33 %). So the quaternion does carry real motion information (a static board has a constant quaternion, and Horizontal and Vertical rotate the board differently), but only the statistics that ignore its absolute value can be trusted. The transfer in the opposite direction (logs → Task 1 recordings) reaches only 66 %, so this result is not complete either. Every window test set here has only 41 to 60 windows.

Tasks 4 and 5: Saved models and code

- **decision_tree_quaternion.json** (3.2 KB), with decision_tree_quaternion.c/.h: the QUAT tree. The generated C code was compiled and compared with the Python tree on all 6,273 logged samples and gave the same class for every one.
- **decision_tree_quaternion_fe.json** (4.7 KB), with decision_tree_quaternion_fe.c/.h: the QUAT+FE tree, plus the window settings, the 24 candidate features, the selected features, and the RFE ranks and ANOVA F-statistics. The generated C code (which calculates the statistics of a 42-sample window and runs the tree) matched the Python model on 684 windows.
- **Scripts:** model_quaternion.py (Task 4), model_quaternion_fe.py (Task 5), quaternion_robustness_check.py (robustness checks of Tasks 4 and 5) and comparison_analysis.py (four-model comparison).

Feature selection code (Task 5)

```
F, p = f_classif(X_train, y_train)                          # ANOVA F-statistic of the 24
features
rfe = RFE(RandomForestClassifier(n_estimators=200, random_state=42),
          n_features_to_select=1, step=1).fit(X_train, y_train)    # rank all 24 features
rank = pd.Series(rfe.ranking_, index=ALL_FEATURES)
selected = list(rank.sort_values().index[:8])                      # keep the top K = 8
model = DecisionTreeClassifier(criterion="gini", max_depth=None, min_samples_leaf=1,
                               min_samples_split=2, random_state=42).fit(train[selected],
y_train)
```

Task 6: Comparison and Insights

Task 6 compares the four decision trees of Tasks 2 to 5 (RAW, RAW+FE, QUAT and QUAT+FE). They were trained on the same recordings, with the same chronological split and the same default hyperparameters, so the comparison is direct. The script comparison_analysis.py reads the results of the four tasks, repeats a robustness test for all four models with one protocol, and produces the tables and charts below.

Comparison of all four models

Table 45. Summary of the four models (same recordings, split and hyperparameters)

Metric	RAW	RAW+FE	QUAT	QUAT+FE
Input to the tree	1 sample	42-sample window	1 sample	42-sample window
Features used (candidates)	6	36	4	8 of 24
Test items	942	41	942	41
Train accuracy	100.00 %	100.00 %	100.00 %	100.00 %

Metric	RAW	RAW+FE	QUAT	QUAT+FE
Validation accuracy	99.68 %	100.00 %	100.00 %	100.00 %
Test accuracy	99.36 %	100.00 %	75.48 %	80.49 %
Gap (train – test) [points]	0.64	0.00	24.52	19.51
Test macro-F1	0.994	1.000	0.723	0.793
Test errors	6	0	231	8
Depth	10	2	3	2
Leaves	48	3	4	3
Nodes	95	5	7	5
JSON size [KB]	43.7	4.8	3.2	4.7
Estimated tree size in C [bytes]	855	45	63	45
Estimated MCU tree latency [µs]	1.43	0.29	0.43	0.29
Window buffer [bytes]	0	1008	0	672
Needed before the tree	nothing	36 statistics	sensor fusion	fusion + 8 statistics

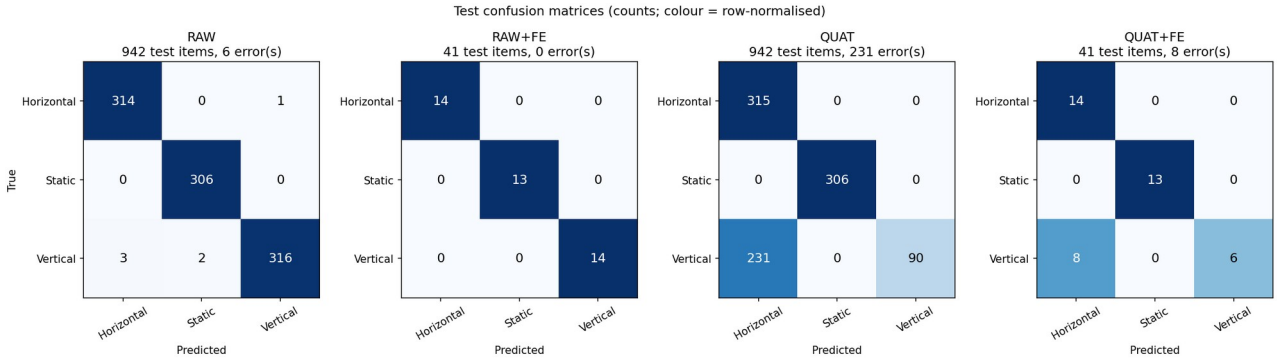


Figure 25. Test confusion matrices of the four models (counts; colour = row-normalised).

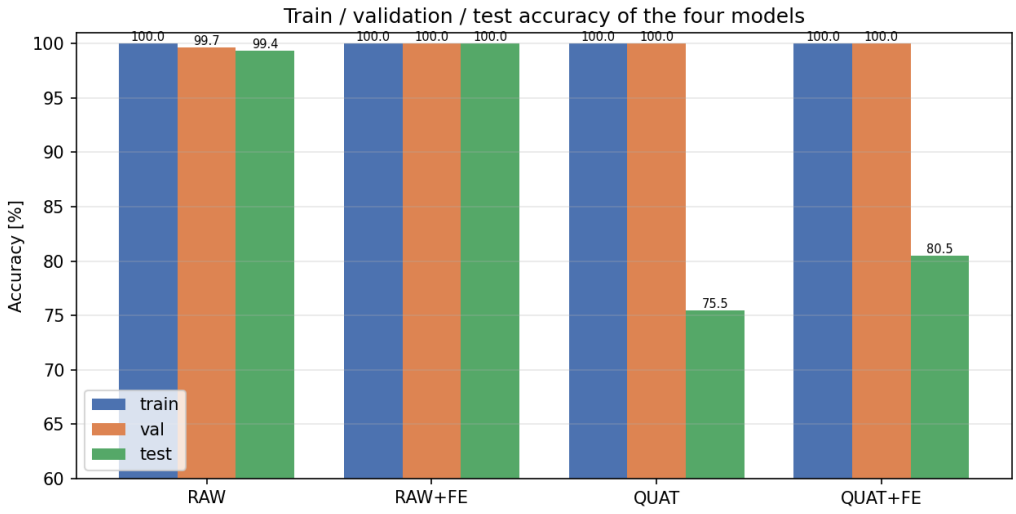


Figure 26. Train, validation and test accuracy of the four models.

Accuracy. The two IMU models are accurate on the test block (RAW+FE 100 % on 41 windows, RAW 99.36 % on 942 samples), while the two quaternion models fail on it (QUAT+FE 80.5 %, QUAT 75.5 %). Their training accuracy is as high as the others (100 %), so only the gap shows the difference: 0 to 0.6 points for the IMU models against 19.5 and 24.5 points for the quaternion models. All the quaternion errors are Vertical samples taken for Horizontal.

Complexity. The quaternion trees are small (7 and 5 nodes) but not smaller than RAW+FE (5 nodes). The size of the tree hardly matters: the trees need about 0.3 to 1.4 µs, while reading the sensor (about 650 µs) and the fusion (about 1,170 µs) are far more expensive in the professor's log. The cost of a model is set by what has to be done before the tree: nothing for RAW, a 1,008-byte buffer and 36 statistics for RAW+FE, and the fusion step for both quaternion models.

Conclusion. On this data the IMU-based models are clearly better than the quaternion-based ones, and RAW+FE and RAW are the most reliable. The quaternion is only useful with motion statistics (motion versus level features, Task 5), and then still needs the fusion step. Advantages of the quaternion representation (few features, a compact tree, orientation already available on the board) do not make up for its dependence on how the board was held.

Second dataset (quaternion-only logs)

Table 46. The same pipelines on the quaternion-only logs (chronological split)

Model	Train	Validation	Test	Depth / leaves	Features used by the tree
QUAT	100.00 %	100.00 %	100.00 %	6 / 7	q0–q3 (4)
QUAT+FE	100.00 %	100.00 %	98.33 %	2 / 3	q0__var, q0__p2p

On these logs both models score 100 % (QUAT) and 98.33 % (QUAT+FE), and the feature selection picks motion features of q0 (variance and peak-to-peak). In this dataset the test block lies within the orientation range of the training block, so the shortcut works. The leave-one-block-out and transfer robustness checks of Tasks 4 and 5 show that this agreement does not mean the models would work in a new session.

Per-class performance

Table 47. Test recall / F1-score per class

Model	Horizontal	Static	Vertical
RAW	0.997 / 0.994	1.000 / 0.997	0.984 / 0.991
RAW+FE	1.000 / 1.000	1.000 / 1.000	1.000 / 1.000
QUAT	1.000 / 0.732	1.000 / 1.000	0.280 / 0.438
QUAT+FE	1.000 / 0.778	1.000 / 1.000	0.429 / 0.600

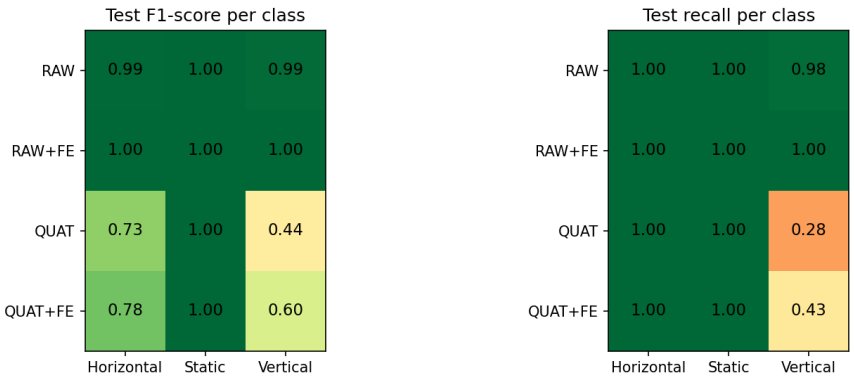


Figure 27. Test F1-score and recall per class.

Static is classified perfectly by all four models and Horizontal almost perfectly (recall at least 0.997). The models differ on Vertical: RAW+FE finds all of it, RAW misses 1.6 % (recall 0.984), while the quaternion models miss most of it (recall 0.28 for QUAT and 0.43 for QUAT+FE) because the quaternion of the Vertical recording drifted between the training block and the test block (Task 4).

Robustness to unseen time blocks (all four models)

The chronological test block is a single piece of each recording, and the window models are tested on only 41 windows. To compare the models more fairly, every recording was cut into 5 consecutive blocks and each block was held out in turn, with the same protocol for all four models. QUAT+FE repeats its ANOVA/RFE selection inside every fold, on the training blocks only.

Table 48. Accuracy on the held-out block (leave-one-time-block-out)

Model	Block 1	Block 2	Block 3	Block 4	Block 5	Mean	Worst
RAW	91.9 %	99.8 %	99.9 %	99.8 %	99.5 %	98.2 %	91.9 %
RAW+FE	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %
QUAT	89.3 %	100.0 %	100.0 %	100.0 %	81.6 %	94.2 %	81.6 %
QUAT+FE	57.1 %	100.0 %	100.0 %	100.0 %	100.0 %	91.4 %	57.1 %

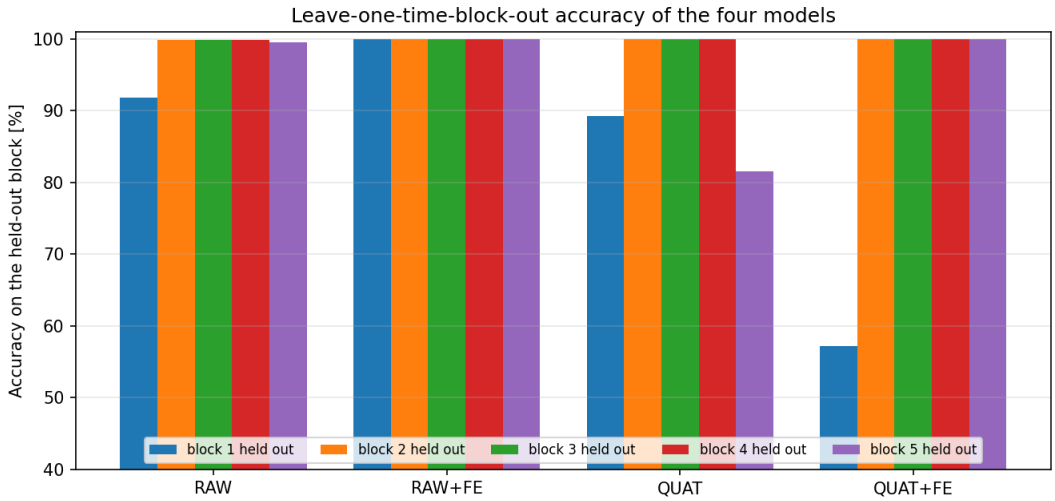


Figure 28. Leave-one-time-block-out accuracy of the four models.

RAW+FE is perfect in all five folds. RAW has a mean of 98.2 % and a worst fold of 91.9 % (first block). The quaternion models fail only in the blocks at the ends of the recordings: QUAT in the first and the last block (89.3 % and 81.6 %) and QUAT+FE in the first block (57.1 %), while blocks 2 to 4 are classified perfectly. This is consistent with the explanation of Tasks 4 and 5: a quaternion tree works for orientations it has seen and cannot extrapolate to orientations outside the training range. The ranking of the two quaternion models also flips between the two tests (QUAT+FE is better on the chronological test block, 80.5 % against 75.5 %, and worse on average over the blocks, 91.4 % against 94.2 %), so they cannot be ranked reliably with this data.

Accuracy against cost

Table 49. Cost of the four models (estimates)

Model	Input	Inputs used by the tree	Tree table [bytes]	Window buffer [bytes]	Total memory [bytes]	Tree time on MCU [µs]	Decision rate	First decision after
RAW	1 sample	6	855	0	855	1.43	every sample	1 sample
RAW+FE	42-sample window	2	45	1008	1053	0.29	every 21 samples	42 samples
QUAT	1 sample	2	63	0	63	0.43	every sample	1 sample
QUAT+FE	42-sample window	2	45	672	717	0.29	every 21 samples	42 samples

Estimates: 9 bytes per tree node, a window buffer of 42 samples × channels × 4 bytes, and 12 cycles per tree level at 84 MHz. They are not measurements on the board.

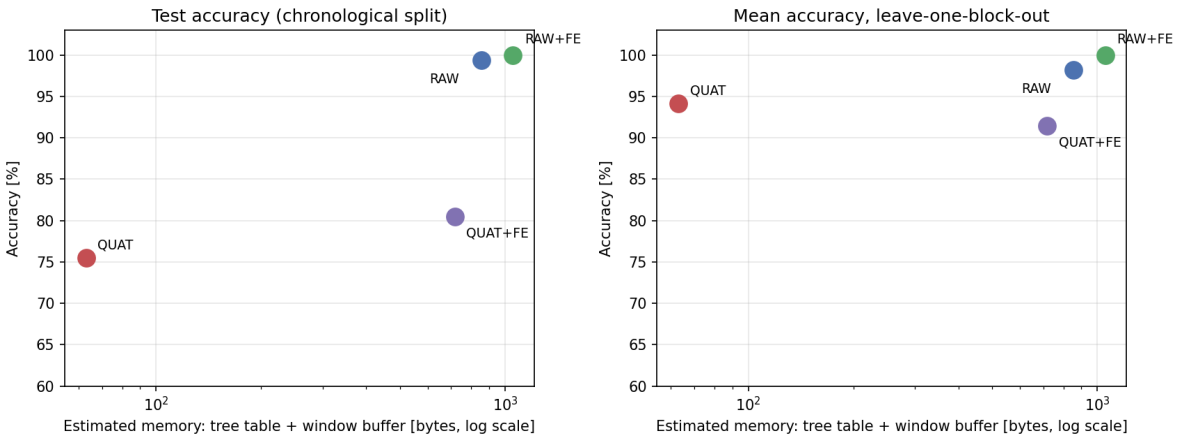


Figure 29. Accuracy against estimated memory: test block (left) and leave-one-block-out mean (right).

The quaternion tree is the smallest model (63 bytes) but the least accurate, and RAW+FE has the highest memory estimate (1,053 bytes, of which 1,008 bytes are the window buffer) and the highest accuracy. The memory differences are small in absolute terms. The time of the trees is also negligible: at most about 1.4 microseconds per decision, against about 650 microseconds for reading the sensor and about 1,170 microseconds for the sensor-fusion step in the instructor's firmware log. The cost of a model is therefore set by what has to be done before the tree: nothing for RAW, a buffer and 36 statistics for RAW+FE, the fusion step for QUAT, and fusion, buffer and statistics for QUAT+FE.

Table 50. Inputs used by each tree (share of the Gini importance)

Model	Features used by the tree
RAW	ACC_Y 46 %, ACC_X 32 %, GYR_Z 20 %, others 2 %
RAW+FE	ACC_X__p2p 50 %, ACC_X__energy 50 %
QUAT	q3 57 %, q2 43 %
QUAT+FE	q3__max 51 %, q1__energy 49 %

The IMU trees rely on the x axis of the accelerometer (and y and the z rotation for RAW), the axes along which the Horizontal movement happens. The quaternion trees rely on q3, the component that carries the orientation difference between the recordings (Table 28), and in QUAT+FE on level statistics (q3__max), not on motion statistics.

Insights

1. **IMU trees generalise, quaternion trees do not.** On the chronological test block the IMU models reach 99.36 % (RAW) and 100 % (RAW+FE) with generalisation gaps of 0.64 and 0 points, while the quaternion models reach 75.48 % (QUAT) and 80.49 % (QUAT+FE) with gaps of 24.5 and 19.5 points.
2. **Windowing helps.** RAW+FE removes the six single-sample errors of RAW, is perfect in all five blocks, and needs a tree of only 5 nodes (two features). Its price is a 1,008-byte buffer and a delay of 42 samples (about 0.59 s) before the first decision.
3. **A quaternion describes orientation, not motion.** Each activity was recorded at its own, drifting orientation, so the trees learned orientation thresholds that fail when the orientation moves. With

only motion statistics (variance and peak-to-peak) the quaternion window model reaches 100 % on the same test windows (Task 5).

4. **Feature selection on one block can pick the wrong features.** ANOVA and RFE chose level statistics of q3 and q1 because they separate the classes in the training block; this is why QUAT+FE has the worst block (57.1 %).
5. **Model size is not the bottleneck.** All trees take at most about 1.4 µs per decision (estimate); the sensor read and the fusion step cost hundreds of times more.
6. **The accuracy numbers are coarse.** The window test sets have 41 windows, so one error is 2.4 points; the block-wise test is more informative than the single test block.

Recommendation

For this data and this hardware, RAW+FE is the best choice: it has the best accuracy on every test, the most robust behaviour across blocks and a tree of only five nodes, provided that a 1,008-byte buffer, a first delay of about 0.59 s and a decision every 0.29 s are acceptable. If every sample must be classified immediately, RAW is the next best (mean block accuracy 98.2 %, no buffer, an estimated 1.4 µs per decision). The quaternion models should not be used on their own: they are small but unreliable, and they need a sensor-fusion step. A quaternion model becomes viable only with motion statistics and with recordings that vary the board orientation.

Implication for the ISPU part of the assignment: a decision tree that takes a quaternion from a Madgwick filter as its input would inherit the same orientation weakness, so its features should be motion statistics of the quaternion (variance, peak-to-peak) or the IMU statistics of RAW+FE, and it should be tested on recordings with different board orientations.

4. Dependent Tasks: Comparison with the Previous Task

Tasks 2 to 5 build on the previous ones. Table C1 states what changed from each task to the next, the change in the results, and the issues or shortcomings. The detailed comparisons are in the task sections (Tables 24 and 27 for Task 3 against Task 2, Table 34 for Task 4 against Task 2, Table 42 for Task 5 against Task 3) and in the comparison of all four models in Task 6 (Table 45).

Table C1. What changed from each task to the next

Comparison	What changed	Result	Issues and shortcomings
Task 2 vs Task 1	Task 2 uses the Task 1 recordings, but reads the raw CSV files (untrimmed, 6,273 samples) and splits each recording chronologically, instead of the trimmed, stratified random split of Task 1 (5,412 samples).	RAW tree: 99.36 % test accuracy on 942 samples; depth 10, 48 leaves, 43.7 KB.	The two preprocessing paths differ (trimming and split), so the sample counts of Task 1 and Task 2 differ. One common pipeline would be cleaner.
Task 3 vs Task 2	Input changes from 1 sample (6 raw values) to a 42-sample window with 36 statistics.	Test accuracy 99.36 % → 100 % (41 windows); depth 10 → 2; leaves 48 → 3; JSON 43.7 → 4.8 KB.	Needs a 1,008-byte buffer, 36 statistics per window and about 0.59 s before the first decision. Only 41 test windows.

Comparison	What changed	Result	Issues and shortcomings
Task 4 vs Task 2	Features change from 6 raw IMU values to the 4 quaternion components.	Test accuracy 99.36 % → 75.48 %; depth 10 → 3; JSON 43.7 → 3.2 KB.	Much worse: the orientation drifts during the recordings (Vertical q3 +0.038 → −0.015); the quaternion also needs the fusion step.
Task 5 vs Task 3 (and Task 4)	36 IMU statistics become 8 of 24 quaternion statistics, selected with ANOVA and RFE.	100 % → 80.49 % (Task 4: 75.48 %, an improvement of 5.0 points); depth 2 → 2; window buffer 1,008 → 672 bytes.	Same drift failure, because the tree uses level features. With motion-only statistics the window model reaches 100 %.

5. Conclusion

What was achieved. Three activities were recorded with the LSM6DSV16X, explored and split (Task 1), and four decision-tree models were trained, evaluated and compared on the same data and with the same hyperparameters: RAW (Task 2), RAW+FE (Task 3), QUAT (Task 4) and QUAT+FE (Task 5, with ANOVA and RFE feature selection). Every model was exported as JSON and as C code, and the generated C code reproduced the Python predictions exactly (6,273 samples for the sample models, 881 and 684 windows for the window models). The IMU-based models were accurate (RAW 99.36 %, RAW+FE 100 % with a tree of depth 2), and the quaternion models were not (QUAT 75.48 %, QUAT+FE 80.49 %). A robustness analysis traced the quaternion failure to the orientation of the board: the orientation differs between the recordings and drifts inside them, and the trees learned thresholds on that level. Using only motion statistics (variance and peak-to-peak) of the quaternion, the window model reached 100 % on the same test windows.

Key issues and shortcomings. Only about 29 s per class were recorded, from one person and in one session, so the window test sets contain only 41 to 60 windows and the percentages are uncertain. Orientation and activity are confounded, because every activity was recorded at its own orientation. The preprocessing of Task 1 (trimmed, stratified random split) differs from that of Tasks 2–5 (untrimmed, chronological split). Latency and model sizes are estimates (9 bytes per node, 12 cycles per tree level at 84 MHz) and host timings, not measurements on the board, and no on-board results are included. The component order of the quaternion in the logs has not been verified. The RAW models contain the gravity direction in the acceleration, so they may depend on the orientation too, which was not tested.

Improvements and next steps. Record several independent sessions per class, with changed board orientation and different people, and test on whole recordings. Run the generated C code on the NUCLEO board and report the measured latency and memory. Use a window of 1.5 s as in the instructor's firmware. Select features with a criterion that rewards robustness across sessions, for example the motion-only statistics. Use one common preprocessing pipeline for all tasks.

6. References

[1] STMicroelectronics, "LSM6DSV16X: 6-axis IMU with embedded sensor fusion, AI, Qvar for high-end applications", datasheet DS13510, https://www.st.com/en/mems-and-sensors/lsm6dsv16x.html

[2] STMicroelectronics, "AN5763: LSM6DSV16X, 6-axis IMU with embedded sensor fusion, AI, Qvar for high-end applications", application note.

[3] STMicroelectronics, “UM3239: Getting started with the X-NUCLEO-IKS4A1 motion MEMS and environmental sensor expansion board for STM32 Nucleo”, user manual, and X-NUCLEO-IKS4A1 data brief, https://www.st.com/resource/en/user_manual/um3239-getting-started-with-the-xnucleoiks4a1-motion-mems-and-environmental-sensor-expansion-board-for-stm32-nucleo-stmicroelectronics.pdf

[4] STMicroelectronics, “UM1724: STM32 Nucleo-64 boards (MB1136)”, user manual (NUCLEO-F401RE pin mapping, ST-LINK virtual COM port), https://www.st.com/resource/en/user_manual/um1724-stm32-nucleo64-boards-mb1136-stmicroelectronics.pdf

[5] STMicroelectronics, MEMS Studio 2.4.1 (Sensor Evaluation, Advanced Features with the Machine Learning Core tools, AlgoBuilder), software.

[6] Zephyr Project, X-NUCLEO-IKS4A1 shield devicetree overlay (I2C address 0x6B of the LSM6DSV16X, interrupt pins), boards/shields/x_nucleo_iks4a1, https://docs.zephyrproject.org/latest/samples/shields/x_nucleo_iks4a1/standard/README.html

[7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python”, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Python libraries pandas, NumPy and Matplotlib.

[8] Course material: instructor’s demonstration video, https://youtu.be/ZS40FpTLe-4 (firmware project DT_QUAT_LSM6DSV16X on the NUCLEO-F401RE, 1.5 s windows, latency measurements).

[9] [Code sources: state the origin of the base scripts imu_tree_pipeline.py, imu_feature_rfe_pipeline.py, quaternion_tree_pipeline.py and quaternion_feature_rfe_pipeline.py (who provided them), as your course requires.]

[10] [If your course requires it, state any use of an AI assistant (for code revision and drafting of this report).]

Appendix A: Code, Modifications and Justifications

For each task, the key code is shown with the lines that implement or modify the key behaviour highlighted in yellow, the comments of the source files, and the justification of each modification. The base code for Tasks 2–5 consists of the four scripts listed in the references; Task 1 has no base code, so its script is included in full in the repository. The complete scripts are in /src of the repository.

Table A1. Final scripts and the base scripts they come from

Final script (in /src)	Task	Derived from (base code in /src/base_code)	Status
data_preparation.py	1	none (no base code)	new
model_raw.py	2	imu_tree_pipeline.py	renamed, unchanged
model_raw_fe.py	3	imu_feature_rfe_pipeline.py	modified
model_quaternion.py	4	quaternion_tree_pipeline.py	small additions
model_quaternion_fe.py	5	quaternion_feature_rfe_pipeline.py	modified
comparison_analysis.py	6	none	new
quaternion_robustness_check.py	4-6	none	new (analysis)

Task 1

Task 1 - loading, labelling and trimming (data_preparation.py)

Status: New code (no base code was provided for Task 1)

```
LABEL_MAP = {"horizontal": "hmotion", "vertical": "vmotion", "static": "static"}

        word = re.split(r"[_.\-\s]", path.stem)[0].lower()
        df["label"] = LABEL_MAP.get(word, word)

        keep = (df["t"] >= TRIM_SECONDS) & (df["t"] <= df["t"].iloc[-1] - TRIM_SECONDS)
        df = df[keep].copy()
```

Justification:

- The class label is taken from the file name and mapped to the names used on the board (hmotion, vmotion, static), so the recordings need no manual labelling.
- The first and last 2 s of each recording are removed to discard the moments of pressing Start and Stop.

Task 1 - stratified 70/15/15 split (data_preparation.py)

Status: New code

```
def stratified_split(data):
    train, temp = train_test_split(
        data, test_size=VAL_FRAC + TEST_FRAC,
        stratify=data["label"], random_state=RANDOM_STATE)
    val, test = train_test_split(
        temp, test_size=TEST_FRAC / (VAL_FRAC + TEST_FRAC),
        stratify=temp["label"], random_state=RANDOM_STATE)
```

Justification:

- Two calls of train_test_split with stratify=label give the required 70/15/15 split with the same class proportions in every set; the fixed seed makes the split reproducible.

Task 2

Task 2 - chronological split of each recording (model_raw.py; base code imu_tree_pipeline.py)

Status: Base code, used unchanged

```
def split_contiguous_recording(frame: pd.DataFrame, train_fraction: float, validation_fraction: float):
    """Split each recording chronologically, so neighboring rows are not randomly mixed."""
    n = len(frame)
    train_end = int(n * train_fraction)
    validation_end = int(n * (train_fraction + validation_fraction))
    if train_end < 1 or validation_end <= train_end or validation_end >= n:
        raise ValueError(f"Not enough rows ({n}) to create non-empty train/validation/test chunks.")
    pieces = {
        "train": frame.iloc[:train_end].copy(),
        "validation": frame.iloc[train_end:validation_end].copy(),
        "test": frame.iloc[validation_end:].copy(),
    }
    for split_name, piece in pieces.items():
        piece["split"] = split_name
    return pieces
```

Justification:

- Included from the base code without modification. Each recording is cut into consecutive blocks (first 70 % train, next 15 % validation, last 15 % test), so almost identical neighbouring samples are not spread over several sets.

Task 2 - the decision tree (model_raw.py; base code imu_tree_pipeline.py)

Status: Base code, used unchanged

```
    model = DecisionTreeClassifier(
        criterion=args.criterion,
        max_depth=args.max_depth,
        min_samples_leaf=args.min_samples_leaf,
        min_samples_split=args.min_samples_split,
        random_state=args.random_state,
    )
    model.fit(X_train, y_train)
```

Justification:

- The hyperparameters are the defaults (gini, no depth limit, min_samples_leaf 1, min_samples_split 2, seed 42). They are kept identical in all later tasks.

Task 3

Task 3 - original: RFE selection (imu_feature_rfe_pipeline.py)

Status: Base code - REMOVED in the modified script

```
    selector = RFE(estimator=base_tree, n_features_to_select=args.n_features, step=args.rfe_step)
    selector.fit(x_train_all, y_train)
    selected_indices = np.flatnonzero(selector.support_).astype(int).tolist()
    selected_names = [ALL_FEATURE_NAMES[i] for i in selected_indices]
    model = selector.estimator_  # final DecisionTreeClassifier already fit by RFE
    class_names = [str(c) for c in model.classes_]
```

Justification:

- Task 3 asks for a tree trained on engineered features, without selection (selection is Task 5). RFE with a single decision tree also degenerated: the features it kept were the last columns in column order. The block was removed.

Task 3 - modified: the six statistics (model_raw_fe.py)

Status: Modified code (highlighted)

```
STATISTICS = ["mean", "var", "max", "min", "p2p", "energy"]
ALL_FEATURE_NAMES = [f"{axis}__{stat}" for axis in RAW_CHANNELS for stat in STATISTICS]  # 36

def extract_window_features(window: np.ndarray) -> dict[str, float]:
    """window: (n_samples, 6) raw values -> 36 features.
    For each axis: mean, variance (population, ddof = 0), maximum, minimum,
    peak-to-peak (max - min) and energy (mean of the squared values)."""
    if window.ndim != 2 or window.shape[1] != len(RAW_CHANNELS):
        raise ValueError("Window must have shape (window_size, 6).")
    x = np.asarray(window, dtype=np.float64)
    mean = x.mean(axis=0)
    var = x.var(axis=0)
    mx, mn = x.max(axis=0), x.min(axis=0)
    energy = (x * x).mean(axis=0)
    out: dict[str, float] = {}
    for i, axis in enumerate(RAW_CHANNELS):
        for stat, value in zip(STATISTICS, (mean[i], var[i], mx[i], mn[i], mx[i] - mn[i], energy[i])):
            out[f"{axis}__{stat}"] = float(value)
    return out
```

Justification:

- The original used 11 statistics (66 features). The assignment lists six: mean, variance, maximum, minimum, peak-to-peak and energy, so 6 axes x 6 statistics = 36 features.
- Energy is the mean of the squares, so it does not depend on the number of samples in a window; the variance divides by n.

Task 3 - modified: training on all 36 features (model_raw_fe.py)

Status: Modified code (highlighted)

```
    model = DecisionTreeClassifier(criterion=args.criterion, max_depth=args.max_depth,
                                   min_samples_leaf=args.min_samples_leaf,
                                   min_samples_split=args.min_samples_split,
                                   random_state=args.random_state)
    model.fit(train_df[ALL_FEATURE_NAMES].to_numpy(dtype=np.float64), train_df["label"].astype(str))
    class_names = [str(c) for c in model.classes_]
```

Justification:

- The tree is fitted directly on all engineered features, with the same hyperparameters as Task 2 (the defaults of both scripts are identical).

Task 4

Task 4 - modified lines in model_quaternion.py (base code quaternion_tree_pipeline.py)

Status: Base code with small additions (highlighted)

```
        out=Path(args.output_dir); out.mkdir(parents=True,exist_ok=True); rows=[]; preds=[]; accs={}
            y=df.label.astype(str); pred=model.predict(df[FEATURES]); acc=accuracy_score(y,pred);
accs[k]=acc
```

```
        jp=out/'decision_tree_quaternion.json'; export_json(model,jp,args);
cp,hp=generate_c(jp,out/'decision_tree_quaternion')
        add_report_outputs(model,splits,accs,out,jp,cp,hp,summary,args)

def add_report_outputs(model,splits,accs,out,jp,cp,hp,summary,args):
    """Accuracy/gap table, importance plot, depth profile, latency estimates (added for the report)."""
    train_acc,val_acc,test_acc=accs['train'],accs['validation'],accs['test']
```

Justification:

- The accuracies of the three sets are kept so that the generalisation gap (train minus test) can be reported.
- The JSON and C files get the names the assignment expects (decision_tree_quaternion.json).
- add_report_outputs() writes the accuracy table, the importance plot, the depth profile and the latency figures. The quaternion columns, the chronological split and the hyperparameters are unchanged.

Task 5

Task 5 - original: selection with a decision tree (quaternion_feature_rfe_pipeline.py)

Status: Base code - REPLACED in the modified script

```
selector=RFE(estimator=DecisionTreeClassifier(random_state=args.random_state),n_features_to_select=args
.n_features,step=args.rfe_step)
        selector.fit(xtr,ytr); selected=[f for f,keep in zip(ALL_FEATURES,selector.support_) if keep];
idx=[ALL_FEATURES.index(f) for f in selected]
model=DecisionTreeClassifier(criterion=args.criterion,max_depth=args.max_depth,min_samples_leaf=args.mi
n_samples_leaf,random_state=args.random_state)
        model.fit(split['train'][selected].to_numpy(float),ytr)
```

Justification:

- RFE used a single decision tree: one feature separates the training windows, so 22 of the 24 features have importance 0 and RFE removes them in plain column order. The statistics were also 11 per component and no ANOVA F-statistic was computed.

Task 5 - modified: statistics, ANOVA and RFE (model_quaternion_fe.py)

Status: Modified code (highlighted)

```
STATS = ["mean", "var", "max", "min", "p2p", "energy"]               # order used by the C code
ALL_FEATURES = [f"{ch}__{s}" for ch in CHANNELS for s in STATS]       # 24 candidates

def anova_table(X, y):
    F, p = f_classif(X, y)
    return pd.DataFrame({"feature": ALL_FEATURES, "anova_F": F, "p_value": p})

def rfe_rank_all(X, y, seed):
    sel = RFE(RandomForestClassifier(n_estimators=200, random_state=seed), n_features_to_select=1,
step=1)
    sel.fit(X, y)
    return pd.Series(sel.ranking_, index=ALL_FEATURES)
```

Justification:

- The statistics are the six of Task 3 (mean, var, max, min, p2p, energy) per quaternion component: 4 x 6 = 24 candidate features, comparable with Task 3.

- The ANOVA F-statistic (f_classif) of all 24 features is computed on the training windows, as the assignment requires.
- RFE uses a random forest, which spreads importance over many features, so the elimination order is informative.

Task 5 - modified: selection and the final tree (model_quaternion_fe.py)

Status: Modified code (highlighted)

```
        anova = anova_table(Xtr, ytr)
        rank = rfe_rank_all(Xtr, ytr, args.random_state)
        selected = list(rank.sort_values().index[:args.n_features])
        model = fit_tree(args, tr[selected].to_numpy(float), ytr)

def fit_tree(args, X, y):
    return DecisionTreeClassifier(criterion=args.criterion, max_depth=args.max_depth,
                                  min_samples_leaf=args.min_samples_leaf,
                                  min_samples_split=args.min_samples_split,
random_state=args.random_state).fit(X, y)
```

Justification:

- The selection is made on the training windows only. The final classifier is a separate DecisionTreeClassifier with the Task 2 defaults (the original reused the RFE estimator).

Task 6

Task 6 - robustness of all four models: selection repeated inside every fold (comparison_analysis.py)

Status: New code (highlighted)

```
        wq_tr = pd.concat([qf.window_segment(block_slice(d, k), WINDOW, STRIDE) for _, d in q_rec for k
in range(K_BLOCKS) if k != j], ignore_index=True)
        wq_te = pd.concat([qf.window_segment(block_slice(d, j), WINDOW, STRIDE) for _, d in q_rec],
ignore_index=True)
        rank = qf.rfe_rank_all(wq_tr[qf.ALL_FEATURES].to_numpy(float),
wq_tr["label"].astype(str).to_numpy(), 42)
        sel = list(rank.sort_values().index[:K_SELECT])         # selection on the training blocks only
        p = dt().fit(wq_tr[sel], wq_tr["label"]).predict(wq_te[sel])
        res["QUAT+FE"].append(float(accuracy_score(wq_te["label"], p)))
    return res
```

Justification:

- Every recording is cut into 5 consecutive blocks and each block is held out in turn, with the same protocol for the four models.
- For QUAT+FE the RFE selection is repeated inside each fold, on the training blocks only, so the held-out block never influences the selected features.
- dt() returns the default DecisionTreeClassifier(random_state=42), the same hyperparameters as in Tasks 2-5.

Tasks 4 and 5: robustness check

Robustness check - motion-only versus level features (quaternion_robustness_check.py)

Status: New code

```
SETS = {
    "all 24 features": q5.ALL_FEATURES,
```

```
    "motion only (var, p2p)": [f"{c}__{s}" for c in CH for s in ("var", "p2p")],
    "level only (mean, max, min, energy)": [f"{c}__{s}" for c in CH for s in ("mean", "max", "min",
"energy")],
}
```

Justification:

- The same window models are trained with only variance and peak-to-peak (motion), only mean, max, min and energy (level), or all 24 features, and tested on unseen time blocks and on a second dataset. This shows which statistics survive when the orientation drifts.

TASK 7 | ISPU IMPLEMENTATION OF QUAT+FE

Implementation evidence, model validation, communication, memory, and benchmark status

Status: demonstrated on LSM6DSO16IS ISPU and STM32F401RE; observed activity IDs 0 (Horizontal), 1 (Static), and 2 (Vertical). Quantitative ISPU timing and total power values still require hardware measurements. This appendix is designed to accompany the existing Tasks 1–6 report, not replace it.

1. System and implementation

Subsystem	Implementation / evidence
Hardware	NUCLEO-F401RE + X-NUCLEO-IKS4A1; LSM6DSO16IS with ISPU
IMU / ISPU	±8 g acceleration; ±2000 dps gyro; native sensor/ISPU cycle 208 Hz
Quaternion fusion	Custom calibration-aware six-axis Madgwick C on ISPU; output w,x,y,z (displayed x,y,z,w)
Quaternion export	STM32 AlgoBuilder timer at 120 Hz; measured ~120.48–120.51 Hz in recorded sessions
Classifier	42-sample windows, 21-sample stride; eight variance/peak-to-peak quaternion features; 17-node depth-4 tree
Classes	0 Horizontal; 1 Static; 2 Vertical; 255 not ready
Communication	STM32F401RE polls 64 bytes of ISPU DOUT registers; USART2 921600 8-N-1 binary protocol to MEMS Studio

Architecture: IMU → Madgwick on ISPU → quaternion window & QUAT+FE decision tree on ISPU → 64-byte ISPU DOUT registers → STM32 timer-driven polling → MEMS Studio / UART. The STM32 does not perform Task 7 decision-tree inference.

2. Training and evaluation

Class	120 Hz samples	Duration (s)	Rate (Hz)
Horizontal	3655	30.321	120.51
Static	3653	30.312	120.48
Vertical	3659	30.361	120.48

The original appended log files contained earlier ~100 Hz runs. Only the later ~120 Hz sessions were extracted for training. A chronological per-recording split (60% train, 15% validation, 15% test, with gap segments) was used. This is a within-session evaluation, not independent repeated-trial validation.

Metric	Motion-feature tree	RFE comparison tree
Train accuracy	288/307 (93.81%)	290/307 (94.46%)
Validation accuracy	75/78 (96.15%)	78/78 (100%)
Test accuracy	75/78 (96.15%)	53/78 (67.95%)
Tree nodes / depth / leaves	17 / 4 / 9	7 / 2 / 4

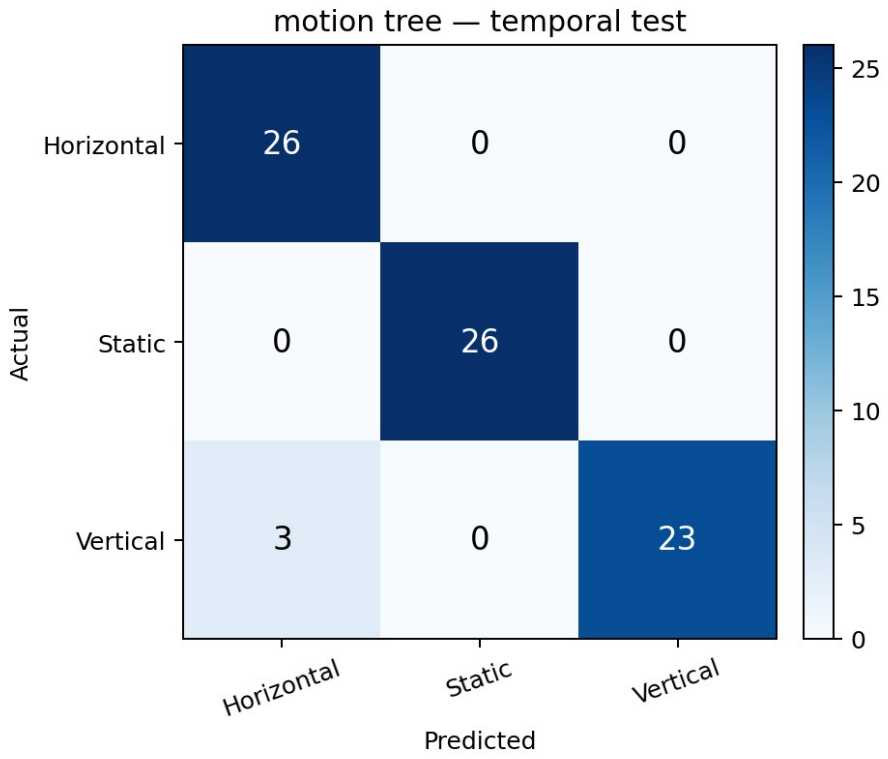


Figure T7-1. Chronological test confusion matrix; axes use model classes 0=Horizontal, 1=Static, 2=Vertical.

The test confusion matrix (true rows, predicted columns) is [[26,0,0],[0,26,0],[3,0,23]]: 75 of 78 windows classified correctly. Because different portions of the same activity recording appear in the splits, these results should not be interpreted as independent hardware accuracy.

3. Python/C reference validation

The bundled desktop checker reproduced the Python/JSON tree output using the generated C feature-extraction and traversal routines on 463 windows: 463/463 matched (100% agreement for that offline check). This establishes numerical equivalence of those implementations on the checked windows, not alignment with predictions captured from separate live ISPU trials.

The tree outputs a leaf-training-purity field as a heuristic quality indicator; it is NOT a calibrated probability of correctness. Fresh ground-truth-labeled hardware trials are still required for a live ISPU-vs-Python confusion matrix and agreement estimate.

4. Source files and compact tree representation

The accompanying package includes native C source and headers: ISPU_208hz/src/main.c, madgwick.c/.h, fusion_core.c/.h, and quat_fe_tree.c/.h, plus the STRED Makefile and sensor configuration. The decision-tree header defines QuatFeNode with feature, left/right children, threshold, predicted class and leaf training purity, and the C array has 17 nodes. Model JSON and training scripts are included for reproducibility.

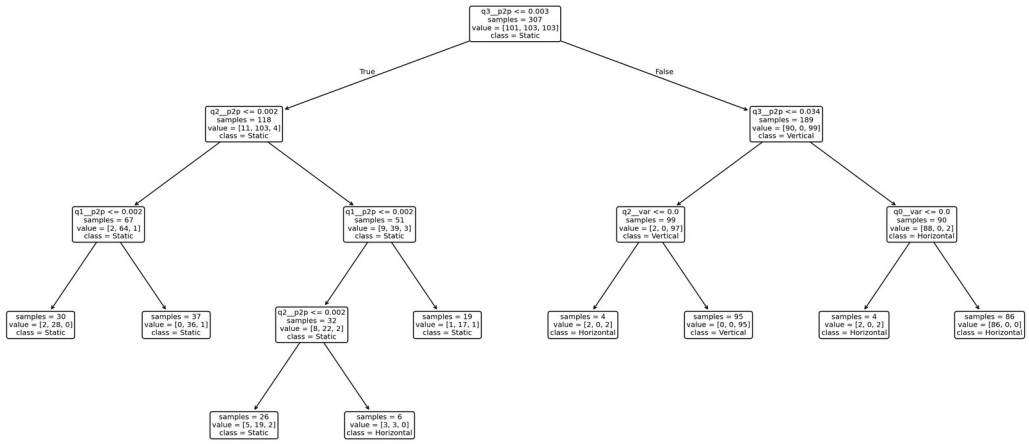


Figure T7-2. Decision-tree structure generated from the 120 Hz QUAT+FE dataset.

5. STM32F401RE reception and UART

The generated STM32 `algo_builder.c` calls Sensor_Hub_Init(0,120,2,3) and ISPU_Init(64). The Sensor_Hub_Handler path in `ab_sensor_hub.c` selects the ISPU register bank and reads all 64 DOUT bytes through the IKS4A1 BSP register API (timer-driven polling). The activity-class ID is extracted at byte offset 40; the UART/display layer sends the selected values to MEMS Studio.

In `com.c`, `Usart_BaudRate = 921600`; `USARTConfig()` configures 8 data bits, 1 stop bit, no parity and no hardware flow control. Actual packets use HAL_UART_Transmit with framed/binary MEMS Studio protocol, not plain-text terminal messages. The corresponding code evidence is bundled.

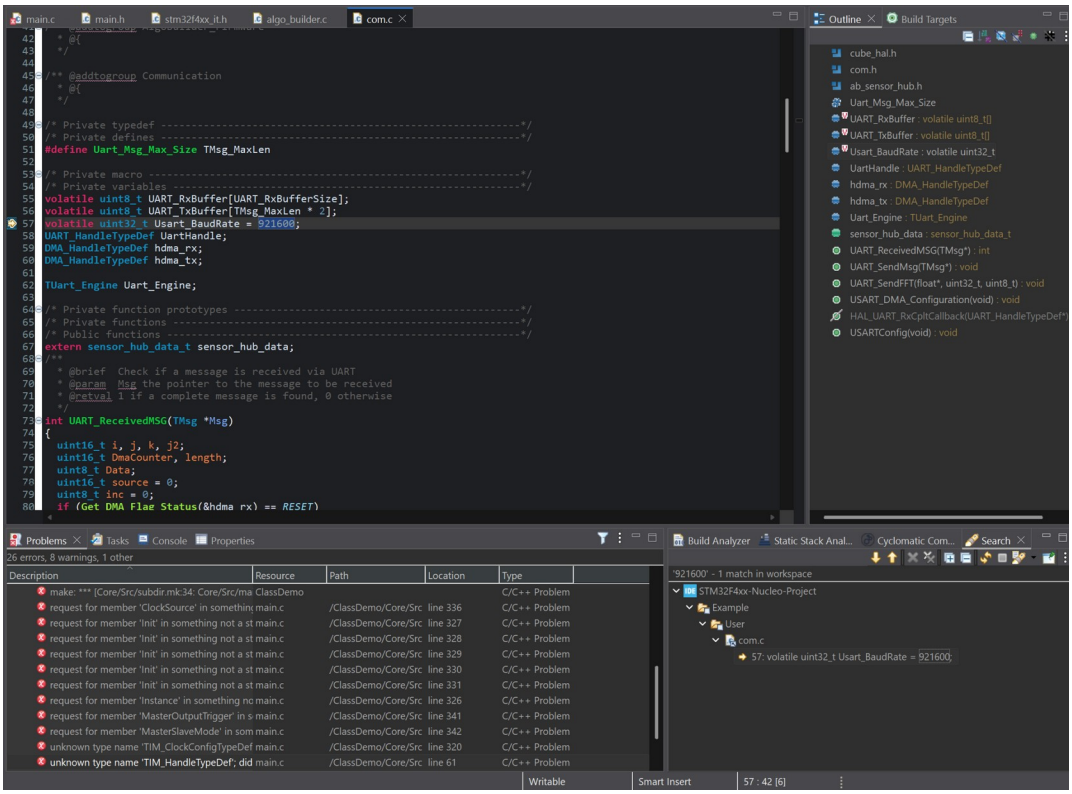


Figure T7-3. STM32CubeIDE code showing the configured 921600 baud value.

6. ISPU memory utilization

Section	Bytes	KiB	Meaning
text	21932	21.418	~66.93% of 32 KiB ISPU program memory
data	108	0.105	Initialized variable section
bss	784	0.766	Uninitialized variable section
data + bss	892	0.871	~10.89% of 8 KiB ISPU data memory
dec	22824	22.289	Sum of ELF sections, not one bank

These are the final STRED `stred-size` figures reported from the successful ISPU build. They exclude complete runtime stack/high-water usage. STM32 FLASH/RAM overhead must be extracted from the STM32 ELF/MAP after building the actual generated project; it is not represented by this ISPU size output.

7. Benchmark table: measured vs. outstanding

Quantity	ISPU implementation	STM32-only reference
Total processing time per sample	NOT MEASURED: INT2 active pulse needed	NOT MEASURED
Decision-tree-only traversal time	NOT ISOLATED	NOT MEASURED
CPU cycles / sample	NOT MEASURED: requires timing and ISPU clock	NOT MEASURED: DWT required
Program/data memory	21.418 / 0.871 KiB	NOT MEASURED: MCU ELF/MAP
Power	Sensor-only baseline below; total unavailable	NOT MEASURED

Power baseline: ST DS13892 specifies approximately 595 µA for combined accelerometer and gyroscope high-performance operation at VDD=1.8 V (ISPU current excluded). Therefore, sensor-only typical baseline power is ~1.071 mW. This is not the measured or estimated total power of the running application. Do not compare this baseline with an STM32-only full-system number.

For total ISPU active time, AN5799 §9.9 describes routing the ISPU sleep indication to INT2; its inactive pulse duration reflects processing time. A benchmark-only conf.txt is supplied. Record measured LOW-pulse widths, ISPU clock and current assumptions in the benchmark worksheet; then compute cycles and duty-cycle-based power. Run an STM32-only reference with DWT instrumentation for an apples-to-apples comparison.

8. Real-time hardware demonstration and limitations

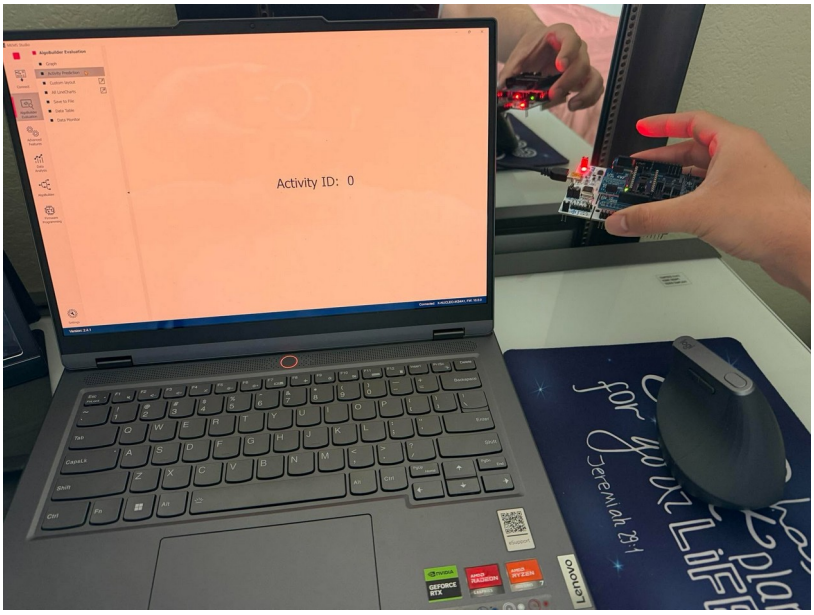


Figure T7-4. Representative live MEMS Studio classification with the physical NUCLEO/IKS4A1 board (Activity ID 0).

Live demonstrations were reported successful for classes 0, 1 and 2; photographs and graph screenshots are included in folder 06_Hardware_Evidence. Observing changing class IDs verifies execution and communication, but not a quantified out-of-sample hardware confusion matrix.

9. Submission status and remaining evidence

Expected output	Status
ISPU C source; compact QUAT+FE model header	INCLUDED
JSON training model; training/validation plots	INCLUDED
Working MEMS Studio native .mems / final plsworkv1.json	NOT AVAILABLE IN PROVIDED FILES — export working design
STM32 polling and UART integration source evidence	INCLUDED; full generated buildable workspace not provided
ISPU and STM32 memory report	ISPU INCLUDED; STM32 memory PENDING
ISPU vs STM32 latency / CPU cycles / power	WORKSHEET INCLUDED; hardware measurement PENDING
Python vs C offline agreement	463/463 INCLUDED
Hardware-vs-reference confusion matrix	PENDING independent labeled sessions
Live screenshots	INCLUDED

References

[1] STMicroelectronics, LSM6DSO16IS datasheet DS13892, Rev. 2, June 2022, https://www.st.com/resource/en/datasheet/lsm6dso16is.pdf

[2] STMicroelectronics, AN5799, LSM6DSO16IS application note, Section 9.9 (INT2 active-time monitoring), https://www.st.com/resource/en/application_note/an5799-lsm6dso16is-alwayson-3axis-accelerometer-and-3axis-gyroscope-with-ispu--intelligent-sensor-processing-unit-stmicroelectronics.pdf

[3] User-provided MEMS Studio 2.4.1 generated source: algo_builder.c, ab_sensor_hub.c, and com.c; final STRED `stred-size` output.

[4] Task 7 model development artifacts: retrain_task7.py, evaluation_results.json, tests/verify_c_matches_python.py, host_validation_output.txt.